Посібник автора Primula

Primula Documenting System 0.4.31 · редакція 11.11.2026

Зміст

Посібник автора Primula....6
Сторінки....6
Що потрібно....6
Як улаштовано документ....7
1. Перший документ....8
1.1. Файл....8
1.2. Заголовок YAML....8
1.3. Текст і абзаци....8
1.4. Заголовки, пункти, списки....9
1.5. Перевірка і збирання....9
1.6. Підсвічування в інших редакторах....9
1.7. Типове оформлення....10
2. Договір....11
2.1. Повністю....11
2.2. Дані — у заголовку, а не в тексті....12
2.3. Початок договору....12
2.4. Розділи і пункти....12
2.5. Посилання на пункти....12
2.6. Суми й числа....13
2.7. Реквізити й підписи....13
2.8. Додаток....13
2.9. Виноски....13
2.10. Повторювані блоки....13
3. Акт і таблиці з підсумками....14
3.1. Проста таблиця....14
3.2. Акт: рядки з даних і підсумок....14
3.3. Таблиця без циклу....15
3.4. Клітинка з кількох абзаців....15
3.5. Оформлення таблиці....15
4. Службовий лист....17
4.1. Приклад....17
4.2. Частини листа....17
4.3. Власні відмінності....18
5. Технічна документація....19
5.1. Застережні блоки й примітки....19
5.2. Варіанти для різних виконань....19
5.3. Скорочення....20
5.4. Покажчик, зміст, переліки....20
5.5. Рисунки....20

5.6. Текст програм та інтерфейс....21
5.7. Спільні фрагменти й ключі....21
6. Проєкт із кількох документів....22
6.1. Файл проєкту....22
6.2. Документ проєкту....22
6.3. Фрагменти: спільний текст....23
6.4. Модулі: макроси й функції....23
6.5. Дані з файлів....23
6.6. Збирання проєкту....24
6.7. Експорт у Markdown, HTML, DOCX....24
6.8. Швидкий перегляд: primula view....25
6.9. Зведений документ....25
6.10. Посилання на інший документ....25
6.11. Карта ключів....25
6.12. Проєкт замовлення....26
7. Стилі....27
7.1. Вибір набору....27
7.2. Застосування стилів у тексті....27
7.3. Власний модуль стилів....28
7.4. Шари....28
7.5. Сторінки й колонтитули....29
7.6. Перехід з ODT Markdown Kit....29
8. Повідомлення транслятора....30
8.1. Як читати повідомлення....30
8.2. Попередження і випуск....30
8.3. Часті помилки....30
8.4. Якщо незрозуміло, що не так....32
9. Паралельні тексти....33
9.1. Приклад....33
9.2. Потоки....33
9.3. Сегменти....34
9.4. Нумерація, посилання, виноски....34
9.5. Перевірка відповідності....35
9.6. Переклад: заготовки, підтвердження, актуальність....35
9.7. Протокол розбіжностей....36
9.8. Стилі....36
10. Код: діалект Python....37
10.1. Де пишеться код....37
10.2. Числа: decimal замість int і float....37
10.3. Рядки....37
10.4. Списки, кортежі, словники....38

10.5. Записи....38
10.6. Дати й довжини....39
10.7. Функції....39
10.8. Глобальні імена: ліниві й незмінні....40
10.9. Модулі й фрагменти....40
10.10. Значення в тексті й макроси....41
10.11. Чого в ядрі немає....42
11. Бібліотеки: розробка й підключення....43
11.1. Бібліотеки комплекту....43
11.2. Що може бібліотека....44
11.3. Будова каталогу....44
11.4. Модуль....45
11.5. Тести....45
11.6. Підключення до проєкту....46
11.7. Дані з датою дії....46
11.8. Що всередині файла: primula describe....47
11.9. Випуск....47
11.10. Команди модуля: |def, param, накопичувач....47
12. Імпорт документів із Word та інших форматів....48
12.1. Команда й вікно Studio....48
12.2. Що переноситься....48
12.3. Як розпізнається структура....48
12.4. Стилі Word....49
12.5. Результат....49
12.6. Що зробити після імпорту....50
Додаток А. Конструкції....51
Команди....51
Короткі форми....53
Маркери закриття....53
Додаток Б. Функції виразів....55
Числа....55
Гроші й ПДВ....55
Прописом....56
Дати й час....56
Одиниці й довжини....57
Текст....57
Списки й записи....57
Дані....58
Математика (std/math)....58
QR-коди й кодування....59
Перевірки й документ....59

Формули таблиць....60
Код стилів....60
Проєкт....61
Методи....61
Додаток В. Ключі YAML-заголовка....63
Додаток Г. Повідомлення транслятора....66
Додаток Д. Стилі комплекту....77
std/styles/default....77
std/styles/business....79
std/styles/letter....80
std/styles/technical....81
std/styles/report....81
std/styles/compact....82
Глоссарий терминов Primula (ru / uk / en)....84

Посібник автора Primula

Посібник для тих, хто пише документи в Primula: договори, акти, листи, інструкції та звіти. Він пояснює, як розв'язати типове завдання, і не повторює довідкових таблиць. Повний перелік конструкцій, функцій, ключів заголовка, повідомлень і стилів наведено в додатках, які генеруються з транслятора й завжди відповідають його версії.

Нормативний документ — специфікація `primula-spec-v1.1-draft.ru.md` і опис стандартної бібліотеки `primula-stdlib-v1.1-draft.ru.md`; українські переклади — `primula-spec-v1.1-draft.uk.md` і `primula-stdlib-v1.1-draft.uk.md`. На початку кожної сторінки посібника в коментарі зазначено розділи специфікації, на які вона спирається.

Сторінки

Сторінка	Про що
Перший документ	файл `.prml`, заголовок YAML, абзаци, заголовки, пункти, перевірка й збирання
Договір	параметри, місце й дата, преамбула, розділи й пункти, посилання на пункти, суми прописом, реквізити, додаток
Акт і таблиці з підсумками	таблиці, стовпці з типами, формули й підсумки, рядки з даних
Службовий лист	набір стилів `letter`: бланк, адресат, номер і дата, підпис
Технічна документація	застережні блоки, умовні варіанти, скорочення, покажчик, рисунки
Проєкт із кількох документів	`project.p.prml`, спільні параметри, фрагменти, модулі, цілі збирання, замовлення
Стилі	набори стилів комплекту, власний модуль стилів, призначення стилів, перехід із Kit
Повідомлення транслятора	як читати повідомлення, часті помилки й що робити
Паралельні тексти	двомовний договір, протокол розбіжностей: потоки й сегменти, мова й нумерація потоку, перевірка відповідності, підтвердження перекладу
Код: діалект Python	числа `decimal`, рядки, колекції, записи, дати, функції, ліниві незмінні глобальні імена, модулі й експорт із фрагмента (`@export`), значення в тексті, чого в ядрі немає
Бібліотеки: розробка й підключення	бібліотека з модулів і даних, що їй доступно й чого немає, будова каталогу, документи-тести з `fail()`, підключення посиланням, копією або підмодулем Git (рішення 136), дані з датою дії, випуск
Імпорт документів	`primula import`: DOCX, ODT, DOC, Markdown, HTML → чернетка Primula; розпізнавання структури, стилі Word, `import-styles.yaml`, що зробити після імпорту

Додатки (генеруються, вручну не правляться): конструкції, функції виразів, ключі заголовка, повідомлення, стилі комплекту. Терміни — за глосарієм.

Що потрібно

- Транслятор `primula` — з пакета або з дерева вихідних кодів (`./primula`). Для PDF потрібен LibreOffice.
- Редактор: Primula Studio (підсвічування, підказки, перевірка, перегляд PDF) або будь-який текстовий редактор з UTF-8.

Studio описано в її довідці (F1 у Studio); посібник показує команди транслятора, які Studio виконує за вас.

Як улаштовано документ

Документ — звичайний текстовий файл. Текст пишеться як є, а все, що стосується розмітки, починається з вертикальної риски `|`: `|<` — заголовок, `|>` — пункт договору, `|(сума)` — значення з коду, `|*{важливо}` — виділення. Транслятор перевіряє файл, обчислює значення, нумерує пункти й посилання та записує документ ODT (а через LibreOffice — PDF). Оформлення задається стилями, а не в тексті: той самий документ друкується інакше, якщо підключити інший набір стилів.

1. Перший документ

1.1. Файл

Документ Primula — текстовий файл у кодуванні UTF-8 з розширенням `.prml`. Поруч із документом можуть лежати файли інших видів; їх розрізняє підрозширення:

Файл	Вид
`договір.prml`	документ — з нього збирається ODT
`умови.f.prml`	фрагмент — частина тексту для вставлення `|include`
`бізнес.m.prml`	модуль — макроси й функції для `|import`
`стилі.s.prml`	модуль стилів
`project.p.prml`	файл проєкту

1.2. Заголовок YAML

Файл починається із заголовка між рядками `---`. У ньому — дані документа, без обчислень:

```
---
doc:
  title: "Довідка про співпрацю"
  language: uk-UA
params:
  організація: "ТОВ «Приклад»"
  рік: 2026
---
|(організація) співпрацює з нами з |(рік) року.
```

- `doc:` — метадані ODT: назва, мова, автор, номер, редакція тощо. У тексті вони доступні як `|(doc.title)`. Мова `uk-UA` визначає лапки, перенесення, формат дат і сум прописом.
- `params:` — параметри документа. У тексті й коді вони доступні за іменем. Під час збирання параметр можна замінити: `primula build довідка.prml --param рік=2027`.

Повний перелік ключів — у додатку «Ключі заголовка».

1.3. Текст і абзаци

Абзаци розділяються порожнім рядком. Перехід на новий рядок усередині абзацу — пробіл, тому довгий абзац можна переносити як зручно.

Усе службове починається з `|`. Те, що йде за рискою, визначає конструкцію:

- літера — команда: `|toc`, `|table(…)`;
- `(` — значення виразу: `|(організація)`;
- розділовий знак — коротка форма: `|<` заголовок, `|>` пункт, `|*{…}` напівжирний;
- друга риска — сама риска: `||`.

Виділення всередині рядка (заповнювач у цьому прикладі дає очікуване попередження P2635):

```
Строк дії — |*{до 31.12.2026}, умови — |/{див. додаток}, код |`{UA-17}.
Назва документа в лапках мови документа: |"{Положення про закупівлі}.
Незаповнене місце в чернетці: |?{прізвище}.
```

Лапки `|"{…}` ставляться за мовою документа («…» для української). Заповнювач `|?{…}` підсвічується й дає попередження під час перевірки: так чернетка не потрапить у випуск непомітно (`--strict` перетворює попередження на помилку).

1.4. Заголовки, пункти, списки

```
|<(number=False) Положення про відрядження
|< Загальні положення
|> Положення визначає порядок направлення працівників у відрядження.
|> Строк відрядження не перевищує 30 календарних днів.
|>> Строк може бути продовжено наказом керівника.
|< Документи
Працівник отримує:
|- посвідчення про відрядження;
|- кошторис витрат.
```

- `|<` — заголовок першого рівня, `|<<` — другого, і так до шостого. Номери ставить транслятор; `number=False` — заголовок без номера (назва документа).
- `|>` — пункт, `|>>` — підпункт. Пункти нумеруються в межах розділу: «1.1.», «1.2.», «1.2.1.» (вигляд номерів задає набір стилів: у звіті за ДСТУ 3008 — без крапки в кінці).
- `|-` — маркований елемент списку, `|#` — нумерований; `|--`, `|##` — наступний рівень.

Пункт, заголовок і елемент списку — рядкові форми: вони закінчуються в кінці рядка. Якщо пункт складається з кількох абзаців, його пишуть оточенням і закривають `|end>`:

```
|>:
Перший абзац пункту.

Другий абзац того ж пункту.
|end>
```

Список і таблиця одразу після пункту входять до нього; звичайний абзац після пункту — уже не частина пункту.

1.5. Перевірка і збирання

```
primula check довідка.prml          # перевірка без запису файлу
primula build довідка.prml          # довідка.odt поруч із вихідним файлом
primula build довідка.prml --pdf    # ще й PDF (потрібен LibreOffice)
```

Перевірка повідомляє про помилки й попередження із зазначенням рядка й стовпця; що з ними робити — на сторінці «Повідомлення транслятора». У Studio ті самі команди — «Перевірити документ» (`Ctrl+Shift+Enter`) і «Зібрати документ» (`F9`); повідомлення відкриває потрібний рядок клацанням.

1.6. Підсвічування в інших редакторах

Файли `.prml` — звичайний текст, їх можна редагувати в будь-якому редакторі. Підсвічування для Kate, KWrite, KDevelop, Midnight Commander (mcedit), Geany і VS Code лежить у каталозі `editors/` пакета: `editors/install.sh` ставить його в профіль користувача (`./install.sh kate vscode` — лише вибрані редактори), `editors/README.md` описує можливості та обмеження кожного редактора. Словник підсвічування будується з довідника транслятора, тому нові команди й функції мови потрапляють до нього під час оновлення комплекту.

1.7. Типове оформлення

Якщо документ не називає стилів, застосовується вбудований набір `std/styles/default`: Liberation Sans 11 pt, поля A4, оформлення заголовків, таблиць, виносок. Для ділових документів, листів, інструкцій і звітів у комплекті є готові набори; їх підключає один рядок заголовка, наприклад `styles: [std/styles/business]`. Докладніше — на сторінці «Стилі».

2. Договір

Сторінка показує, як зібрати договір: дані в заголовку, текст із нумерованими пунктами, посилання на пункти замість набраних вручну номерів, суми прописом, реквізити сторін і додаток. Оформлення — набір стилів `std/styles/business`.

2.1. Повністю

```
---
doc:
  title: "Договір про надання послуг"
  identifier: "П-2026/017"
  language: uk-UA
styles: [std/styles/business]
params:
  місто: "м. Харків"
  дата: 2026-10-12
  строк_днів: 5
  виконавець: {назва: "ТОВ «Виконавець»", код: "00000000", особа: "директора Петренка
Петра Петровича", підписант: "П. П. Петренко"}
  замовник: {назва: "ТОВ «Замовник»", код: "11111111", особа: "директора Іваненка
Івана Івановича", підписант: "І. І. Іваненко"}
---
|code:
послуги = [
    record(назва="Налаштування обладнання", кількість=dec("2"), ціна=dec("1500.00")),
    record(назва="Навчання персоналу", кількість=dec("1"), ціна=dec("2400.00")),
]
сума = sum([п.кількість * п.ціна for п in послуги])
|end
|.DocumentTitle: ДОГОВІР № |(doc.identifier)
|.DocumentSubtitle: про надання послуг
|.DatePlace: |(місто), |(fmt_date(дата, "contract"))

|.Preamble: |(виконавець.назва) (далі — |.DefinedTerm{Виконавець}) в особі |
(виконавець.особа) та |(замовник.назва) (далі — |.DefinedTerm{Замовник}) в особі |
(замовник.особа) уклали цей договір про таке.

|<(id="предмет") Предмет договору
|>(id="послуги") Виконавець надає послуги, перелічені в |@("специфікація", "label"), а
Замовник приймає й оплачує їх.

|<(id="ціна") Ціна і розрахунки
|> Ціна послуг становить |(money_full(сума, "UAH")).
|>(id="оплата") Замовник оплачує послуги протягом |(строк_днів) |(plural(строк_днів,
"банківського дня", "банківських днів", "банківських днів"))|footnote{Банківський день
— день, у який банки України здійснюють розрахунки.} з дати підписання акта.

|<(id="відповідальність") Відповідальність сторін
|> За прострочення оплати (|@("оплата", "label")) Замовник сплачує пеню в розмірі
подвійної облікової ставки НБУ.

|<(number=False) Реквізити і підписи сторін
|table(style="TableParties", header=1):
|: Виконавець |& Замовник
|: |(виконавець.назва) |& |(замовник.назва)
|: Код ЄДРПОУ |(виконавець.код) |& Код ЄДРПОУ |(замовник.код)
|: _____ |(виконавець.підписант) |& _____ |(замовник.підписант)
|end table

|appendix(id="специфікація", title="Специфікація послуг", subtitle=f"до Договору №
{doc.identifier}"):
|table(style="TableServices", header=1, footer=1, cols=["52%", col("12%", id="к",
type="decimal", align="right"), col("18%", id="ц", type="money", align="right"),
```

```
col("18%", id="c", type="money", currency="UAH", align="right")]):
|: Послуга |& Кількість |& Ціна, грн |& Сума, грн
|for(п in послуги):
|: |(п.назва) |& |(п.кількість) |& |(п.ціна) |& |=(this_row("к") * this_row("ц"))
|endfor
|:|&(span=3, align="right") Разом |& |=(sum(column("c", rows=body_rows())))
|end table
|end appendix
```

Далі — частинами.

2.2. Дані — у заголовку, а не в тексті

Усе, що змінюється від договору до договору, записується в `params:` заголовка: сторони, місто, дата, строки. У тексті значення виводяться виразом `|(…)`: `|(місто)`, `|(виконавець.назва)`. Вкладений запис (`виконавець: {назва: …, код: …}`) читається через крапку.

Так текст не треба правити під час зміни сторони, а Studio показує параметри окремою формою. Якщо договорів багато, спільні дані виносять у проєкт (сторінка «Проєкт»).

Не вписуйте в шаблон справжні коди, IBAN і підписи — поки даних немає, використовуйте заповнювач `|?{…}`: перевірка нагадає про нього, а `--strict` не дасть зібрати випуск.

2.3. Початок договору

Назва, підзаголовок, місце й дата, преамбула — абзаци зі стилями набору `business`: `|.DocumentTitle: …`, `|.DatePlace: …`, `|.Preamble: …`. Стиль, записаний на початку рядка через `|.Ім'я:`, стосується всього абзацу цього рядка; `|.Ім'я{…}` — фрагмента всередині рядка (тут `|.DefinedTerm{Виконавець}` виділяє визначуваний термін).

Дата «по-договірному» — функція `fmt_date(дата, "contract")`: «12» жовтня 2026 р. Інші формати дат, а також `fmt_date(дата, "long")` і `"short"` — у додатку «Функції виразів».

2.4. Розділи і пункти

`|<` — розділ, `|>` — пункт, `|>>` — підпункт. Номери ставить транслятор, тому вставлення нового пункту в середину не потребує ручного перенумерування.

2.5. Посилання на пункти

Ніколи не пишіть номер пункту вручну. Пункту чи розділу, на який посилаються, дають `id=`, а в тексті пишуть посилання:

Запис	Текст у документі
`|@("оплата")`	2.2
`|@("оплата", "label")`	п. 2.2
`|@("ціна", "label")`	розд. 2
`|@("специфікація", "label")`	дод. 1
`|@("ціна", "title")`	Ціна і розрахунки

Посилання може стояти раніше за ціль. Якщо пункт вилучено або його `id=` змінено, перевірка повідомить про невідому ціль — завислих посилань у документі не буває. Відмінкова форма мітки — `|@("ціна", "label", case="loc")` → «розділі 2».

2.6. Суми й числа

Гроші в Primula — десяткові числа: пишіть `dec("1500.00")` або параметр заголовка, а не дріб із рухомою комою. Виведення — функціями бібліотеки:

```
|code:
сума = dec("5400.00")
|end
|(money(сума, "UAH")) — |(money_full(сума, "UAH")).
ПДВ 20 %: |(money(vat(сума, 20), "UAH")), разом з ПДВ |(money(with_vat(сума, 20),
"UAH")).
Строк — |(words(5)) |(plural(5, "день", "дні", "днів")).
```

`money` дає «5 400,00 грн», `money_full` — суму цифрами й прописом, `words` — число словами, `plural` — форму слова для числа. Число чи дата, виведені без функції форматування, дають попередження: так у договір не потрапить «5400.0».

2.7. Реквізити й підписи

Реквізити сторін — таблиця без ліній `TableParties` (дві колонки по 50 %): перший рядок — назви сторін, далі — по рядку на реквізит. Рядок підпису — у тій самій таблиці, тому підписи сторін стоять на одному рівні.

2.8. Додаток

`|appendix(…):` … `|end appendix` починає додаток з нової сторінки, виводить «Додаток 1», підзаголовок і назву та відкриває власну нумерацію розділів, рисунків і таблиць. Посилання `|@("специфікація", "label")` дає «дод. 1». Таблицю з підсумком у додатку розібрано на сторінці «Акт і таблиці з підсумками».

2.9. Виноски

`|footnote{текст}` ставить виноску внизу сторінки; номер рахує Writer.

2.10. Повторювані блоки

Якщо блок реквізитів чи підпису повторюється в багатьох договорах, його оформлюють макросом у модулі (`|def`, сторінка «Проєкт») або фрагментом (`|include`). Приклад із комплекту — `examples/02-contract` (модуль `lib/business.m.prml` з макросами реквізитів і підпису).

3. Акт і таблиці з підсумками

Таблиця — оточення `|table(…):` … `|end table`. Рядок починається з `|:`, клітинки розділяє `|&`. Транслятор рахує підсумки за формулами, перевіряє кількість клітинок у рядках і оформлює числа й суми за мовою документа.

3.1. Проста таблиця

```
|table(cols=["40%", "*"], header=1):
|: Показник |& Значення
|: Строк поставки |& 10 робочих днів
|: Місце поставки |& склад Покупця
|end table
```

- `header=1` — один рядок шапки; на наступних сторінках він повторюється.
- `cols` — ширини стовпців: `"40mm"` — фіксована, `"30%"` — частка таблиці, `"*"` — залишок, `"auto"` — за вмістом, `"20mm..60mm"` — за вмістом у межах.
- Кількість клітинок кожного рядка має збігатися з кількістю стовпців; зайва чи відсутня клітинка — помилка з номером рядка. Порожня клітинка — `|&` без тексту.

3.2. Акт: рядки з даних і підсумок

Нехай позиції акта лежать у CSV:

```
назва,кількість,ціна
Налаштування обладнання,2,1500.00
Навчання персоналу,1,2400.00
Виїзд спеціаліста,3,350.00
```

Таблиця акта з підсумком і ПДВ:

```
|code:
позиції = load_csv("data/позиції.csv", types={"кількість": "decimal", "ціна":
"money"})
|end
|table(id="т-акт", caption="Перелік наданих послуг", header=1, footer=3, cols=["10mm",
"*", col("18mm", id="к", type="decimal", align="right"), col("26mm", id="ц",
type="money", align="right"), col("30mm", id="с", type="money", currency="UAH",
align="right")]):
|: № |& Послуга |& К-сть |& Ціна |& Сума
|for(н, п in enumerate(позиції, 1)):
|: |(н) |& |(п.назва) |& |(п.кількість) |& |(п.ціна) |& |=(this_row("к") *
this_row("ц"))
|endfor
|:|&(span=4, align="right") Разом без ПДВ |& |=(sum(column("с", rows=body_rows())))
|:|&(span=4, align="right") ПДВ 20 % |& |=(round(cell(len(позиції) + 2, "с") *
dec("0.20"), 2))
|:|&(span=4, align="right") |*{Усього} |& |=(cell(len(позиції) + 2, "с") +
cell(len(позиції) + 3, "с"))
|end table
Див. |@("т-акт", "label").
```

Що тут відбувається:

- `load_csv` читає файл; `types=` оголошує типи стовпців, інакше все читається рядками. Шлях — відносно документа й не виходить за корінь проєкту. Так само працюють `load_yaml` і `load_json`.

- `col(ширина, id=…, type=…, currency=…, align=…)` описує стовпець. `id` — ім'я стовпця у формулах. `type="money"` виводить значення з двома знаками («1 500,00»), `currency="UAH"` додає «грн», `type="decimal"` — число з розділювачами груп.
- `|for(…):` … `|endfor` повторює рядок для кожної позиції.
- `|=(…)` — формула клітинки. Вона обчислюється, коли відомі всі звичайні клітинки: `this_row("к")` — значення стовпця `к` цього рядка, `column("с", rows=body_rows())` — стовпець за рядками тіла (без шапки й підсумків), `cell(рядок, стовпець)` — клітинка за номером рядка (з 1, рахуючи шапку) і стовпцем.
- `footer=3` — три підсумкові рядки; вони не входять до `body_rows()`.
- `|:|&(span=4, align="right")` — аргументи першої клітинки пишуться впритул після `|:`; `span=4` об'єднує чотири стовпці.
- `caption=` і `id=` дають підпис «Таблиця 1 — Перелік наданих послуг» і ціль посилання «табл. 1».

Гроші рахуються десятковими числами, тому копійки не губляться. Округлення — `round(x, 2)`; правила округлення бухгалтерських сум і ПДВ — функції `round_money`, `vat`, `with_vat` бібліотеки (додаток «Функції»).

3.3. Таблиця без циклу

Якщо рядки — це цілі записи, цикл не потрібен:

```
|code:
склад = [record(назва="Кабель", к=dec("120")), record(назва="Роз'єм", к=dec("40"))]
|end
|table(data=склад, fields=["назва", "к"], titles=["Найменування", "Кількість"],
cols=["*", col("30mm", id="к", type="decimal", align="right")], footer=1):
|:|&(align="right") Разом |& |=(sum(column("к", rows=body_rows())))
|end table
```

3.4. Клітинка з кількох абзаців

Компактний рядок `|:` уміщується в одному рядку вихідного тексту. Для клітинки зі списком чи кількома абзацами — розгорнута форма:

```
|table(cols=["35%", "*"], header=0):
|row:
|cell: Обов'язки Виконавця
|cell:
Виконавець зобов'язується:

|- надати послуги в строк;
|- передати звіт.
|end cell
|end row
|end table
```

Компактні й розгорнуті рядки можна змішувати в одній таблиці.

3.5. Оформлення таблиці

Вигляд таблиці задає її стиль: `|table(style="TableCompact", …)`. У наборах стилів комплекту є готові: `TableGrid` (типовий), `TableCompact`, `TableNoBorders`; у `business` —

`TableServices`, `TableSchedule`, `TableForm`, `TableParties` із заданими ширинами стовпців (власні `cols` таблиці їх замінюють). Разові відмінності — аргументи таблиці, рядка (`|:(background="#EEEEEE")`) і клітинки (`|&(align="center")`): стиль таблиці → стовпець → рядок → клітинка, кожен рівень уточнює попередній. Лінії — `frame=` (рамка) і `rules=` (`"all"`, `"rows"`, `"cols"`, `"none"`).

Таблиця всередині розділу з колонками (`|section(cols=2)`) отримує ширину колонки.

4. Службовий лист

Лист оформлюється набором стилів `std/styles/letter`. Він побудований на діловому наборі `business` і відповідає ДСТУ 4163:2020: поля — ліве 30 мм, праве 10 мм, верхнє й нижнє 20 мм; текст з абзацним відступом 10 мм, адресат — блоком на 90 мм від лівого поля, номер сторінки — з другої сторінки вгорі посередині.

4.1. Приклад

```
---
doc:
  title: "Щодо графіка технічної підтримки"
  language: uk-UA
styles: [std/styles/letter]
params:
  номер: "12/3"
  дата: 2026-10-16
  відправник: {назва: "ТОВ «Виконавець»", адреса: "вул. Сумська, 1, м. Харків, 61000",
телефон: "+380 57 000 00 00", код: "00000000", керівник: "Петро ПЕТРЕНКО"}
  адресат: {посада: "Директору", назва: "ТОВ «Замовник»", особа: "Івану ІВАНЕНКУ"}
---
|.LetterheadName: |(відправник.назва)
|.LetterheadDetails: |(відправник.адреса), тел. |(відправник.телефон)
|.LetterheadDetails: Код ЄДРПОУ |(відправник.код)
|.OutgoingNumber: Вих. № |(номер) від |(fmt_date(дата, "short"))

|.Recipient: |(адресат.посада) |(адресат.назва)
|.Recipient: |(адресат.особа)

|.LetterSubject: |(doc.title)

|.Salutation: Шановний пане Іване!

Повідомляємо, що роботи з налаштування обладнання завершено, акт приймання-передачі
підписано сторонами.

Пропонуємо погодити графік технічної підтримки на листопад — грудень 2026 року. Проєкт
графіка додається.

|.AttachmentLabel: Додаток: проєкт графіка на 1 арк. в 1 прим.

|table(style="TableParties", header=0, cols=["60%", "40%"]):
|: Директор |& |(відправник.керівник)
|end table

|.Executor: Олена Коваленко +380 57 000 00 01
```

4.2. Частини листа

Частина	Стиль	Зауваження
назва й реквізити відправника	`LetterheadName`, `LetterheadDetails`	у лівій частині, рядки до 73 мм
номер і дата	`OutgoingNumber`	«Вих. № … від …»; дата — `fmt_date(дата, "short")`
адресат	`Recipient`	кожен рядок — окремий абзац цього стилю
заголовок до тексту	`LetterSubject`	«Щодо …», без крапки
звертання	`Salutation`	посередині

Частина	Стиль	Зауваження
текст	без стилю	звичайні абзаци отримують `BodyIndented` — набір прив'язує до нього текст
додатки	`AttachmentLabel`	
підпис	таблиця `TableParties`	посада ліворуч, ім'я праворуч на одній лінії
виконавець	`Executor`	дрібним шрифтом у кінці

Перша сторінка листа — стиль сторінки `LetterFirst` без номера; наступні — `Portrait` з номером угорі. Це задає набір, у тексті нічого писати не треба.

4.3. Власні відмінності

Якщо в організації прийнято інший бланк, не правте набір комплекту: зробіть власний модуль над ним і підключіть його замість набору (сторінка «Стилі»):

```
---
kind: styles
extends: std/styles/letter
paragraphs:
  LetterheadName: {size: 13pt, color: "#1F4E79"}
---

---
doc: {title: "Лист", language: uk-UA}
styles: [styles/бланк.s.prml]
---
|.LetterheadName: ТОВ «Приклад»

Текст листа.
```

Логотип на бланку — `|img("assets/логотип.svg", style="LogoSmall")` на початку листа; розміри логотипа й ілюстрацій (`LogoSmall`, `Logo`, `Illustration` …) є в наборі `business`.

5. Технічна документація

Настанови, інструкції, технічні описи оформлюються набором стилів `std/styles/technical` (або `report` — для звітів за ДСТУ 3008). Нижче — засоби, потрібні саме технічним документам: застережні блоки, варіанти для різних виконань виробу, скорочення, покажчик, рисунки.

5.1. Застережні блоки й примітки

`|!("вид")` — смислова розмітка: автор називає вид, а сигнальне слово мовою документа, колір і оформлення беруться зі стилів.

Вид	Слово (uk)	Коли
`danger`	НЕБЕЗПЕКА	неминуча загроза життю
`warning`	ПОПЕРЕДЖЕННЯ	можлива загроза життю чи тяжка травма
`caution`	ОБЕРЕЖНО	можлива легка травма
`notice`	УВАГА	можливе пошкодження майна
`note`	Примітка	пояснення
`tip`	Порада	рекомендація

```
|!("warning", hazard="Висока напруга", consequence="Ураження електричним струмом"):
Перед обслуговуванням від'єднайте пристрій від мережі.

|- Вимкніть автоматичний вимикач.
|- Перевірте відсутність напруги.
|end!

|!("notice") Не встановлюйте пристрій поблизу джерел тепла.

|!("note") Значення наведено для температури 20 °C.
|!("note") Допуск — ±5 %.
```

- Рядкова форма `|!("вид") текст` — блок з одного абзацу; оточення `|!(…):` … `|end!` — з кількох абзаців і списків.
- У `danger` і `warning` потрібен `hazard=` (вид і джерело небезпеки); без нього перевірка попередить.
- Кілька `note` поспіль об'єднуються в групу «Примітки:» з нумерацією.

5.2. Варіанти для різних виконань

Один вихідний текст обслуговує кілька виконань виробу. Властивості виконання перелічуються в параметрі `features`, а умовні частини позначаються `when=`:

```
---
doc: {title: "Настанова з експлуатації", language: uk-UA}
styles: [std/styles/technical]
params:
  модель: "X300"
  features: [wifi]
---
|< Підключення
|> Під'єднайте кабель живлення.
|>(when=has("wifi")) Підключіть пристрій до мережі Wi-Fi.
|>(when=has("bluetooth")) Увімкніть Bluetooth на телефоні.
|table(header=1, cols=["*", "30%"]):
|: Параметр |& Значення
|: Напруга |& 230|~В
```

```
|:(when=модель == "X300") Потужність |& 2,2|~кВт
|end table
|if(has("wifi")):
|< Бездротова мережа
|> Пристрій підтримує стандарт 802.11n.
|endif
```

- `when=` є в будь-якої блокової команди, оточення й рядка таблиці; `|if(…):` … `|endif` — для довільної групи блоків.
- `has("wifi")` істинне, якщо `wifi` є в `features`. Набір властивостей задають заголовок документа, проєкт або ціль збирання проєкту (`params: {features: [wifi, bluetooth]}` у `targets`): так з одного тексту збираються настанови всіх виконань. Параметр командного рядка `--param` списків не приймає.
- Вилучені частини не потрапляють в ODT; нумерація, зміст і посилання рахуються за рештою тексту. Посилання на вилучену частину — помилка.
- `primula build настанова.prml --reviewe` збирає всі варіанти одразу й позначає кожну умовну частину її умовою (зеленим — увімкнена в поточному виконанні, червоним — ні), а на сторінки ставить водяний знак «НЕ ДЛЯ ВИПУСКУ». Так перевіряють увесь текст одним файлом.

5.3. Скорочення

```
|abbr("ПЗ", "програмне забезпечення") встановлюється з диска.
Оновлення |abbr("ПЗ") виконується автоматично.
|list_of("abbr", title="ПЕРЕЛІК СКОРОЧЕНЬ")
```

Під час першого вживання виводиться «програмне забезпечення (ПЗ)», далі — лише «ПЗ». Відмінок розшифрування в цьому місці — `text="програмного забезпечення"`. `|list_of("abbr")` будує перелік скорочень таблицею.

5.4. Покажчик, зміст, переліки

```
|toc(max_level=2, title="ЗМІСТ")
|< Живлення
Напруга|index("напруга живлення") подається через адаптер.
|list_of("index", title="ПОКАЖЧИК")
```

Зміст, переліки рисунків і таблиць (`|list_of("figure")`, `|list_of("table")`) і алфавітний покажчик — оновлювані покажчики Writer; номери сторінок у них заповнює Writer або транслятор під час збирання PDF.

5.5. Рисунки

```
|figure(svg='<svg xmlns="http://www.w3.org/2000/svg" width="40mm" height="20mm"><rect
x="1" y="1" width="38" height="18" fill="none" stroke="black"/></svg>', id="рис-
схема"):
Схема підключення
|end figure
Див. |@("рис-схема", "label").
```

- `|figure(шлях або svg=…, id=…):` … `|end figure` — нумерований рисунок; тіло — підпис («Рисунок 1 — Схема підключення»). `|img(…)` — зображення без номера й підпису.
- Шлях до файлу — відносно документа: `|figure("assets/схема.svg", width="120mm")`. Формати — PNG, JPEG, SVG.

- `svg=` — зображення з рядка SVG, обчисленого кодом (графік, шкала, QR-код із функції модуля).
- Розмір — `width=` або `height=`; розміри за стилем — `style="Illustration"` (набори `business`, `letter`).

5.6. Текст програм та інтерфейс

Набір `technical` дає стилі для елементів інтерфейсу й команд:

```
---
doc: {title: "Інструкція", language: uk-UA}
styles: [std/styles/technical]
---
|list("StepList"):
|# Натисніть |.Key{Ctrl+S}.
|# Виберіть |.MenuItem{Файл → Зберегти} і вкажіть файл |.FileName{звіт.odt}.
|end list
|.StepResult: Файл буде збережено.

|.Preformatted: primula check звіт.prml
```

Символьні `Key`, `MenuItem`, `FileName`, `Command`, `Variable`; абзацні `Preformatted`, `ProgramOutput`, `StepResult`, `DefinitionTerm`/`DefinitionText`; список кроків `StepList`; таблиця `TableRevisions` для аркуша реєстрації змін. Повний перелік — додаток «Стилі комплекту». Текст, який треба зберегти літера в літеру (лістинг із символами `|`), пишуть у `|verbatim:` … `|end`.

5.7. Спільні фрагменти й ключі

Повторювані тексти (загальні заходи безпеки, гарантія) виносять у фрагменти `*.f.prml` і підключають `|include`; у проєкті їх можна підміняти за виконанням через карту ключів `keys:` — сторінка «Проєкт». Повний приклад — `examples/03-manual` і `examples/06-safety`.

6. Проєкт із кількох документів

Коли документів кілька — договір, акти, рахунки, листи однієї компанії, — спільні дані, стилі й повторювані тексти збирають у проєкт. Проєкт — каталог із файлом `project.p.prml`. Транслятор знаходить його сам, піднімаючись від документа вгору (до каталогу з `.git`).

```
договори-2026/
  project.p.prml        параметри, стилі, ключі, цілі збирання
  договір.prml
  акт.prml
  fragments/            тексти для |include (*.f.prml)
  lib/                  макроси й функції для |import (*.m.prml)
  styles/               власний модуль стилів (*.s.prml)
  data/                 CSV, YAML, JSON
```

6.1. Файл проєкту

```
---
kind: project
name: "Договори 2026"
language: uk-UA
styles: [styles/компанія.s.prml]
doc_defaults: {organization: "ТОВ «Виконавець»", creator: "Відділ договорів"}
params:
  місто: "м. Харків"
  виконавець: {назва: "ТОВ «Виконавець»", код: "00000000", директор: "П. П. Петренко"}
  ставка_пдв: 20
targets:
  - {source: договір.prml, output: out/договір.odt, pdf: true}
---
```

- `params:` — спільні параметри. Документ бачить їх за іменем (`|(місто)`) і як `|(project.місто)`. Параметр документа з тим самим іменем замінює значення проєкту для цього документа.
- `doc_defaults:` — типові метадані ODT для всіх документів; ключ `doc:` документа замінює їх поштучно.
- `styles:` — модулі стилів проєкту; вони підключаються до всіх документів.
- `targets:` — цілі пакетного збирання.

Власний модуль стилів проєкту побудовано на наборі комплекту (сторінка «Стилі»):

```
---
kind: styles
extends: std/styles/business
paragraphs:
  Реквізити: {parent: Details, keep_together: true}
---
```

6.2. Документ проєкту

```
---
doc: {title: "Договір про надання послуг"}
---
|(виконавець.назва), |(місто). Ставка ПДВ — |(percent(ставка_пдв)).

|.Реквізити: Код ЄДРПОУ |(виконавець.код)
```

Документ не називає ні стилів, ні мови: усе приходить із проєкту.

6.3. Фрагменти: спільний текст

Фрагмент `*.f.prml` — шматок тексту для вставлення. Параметри фрагмента оголошуються в його заголовку й передаються аргументами `|include`:

```
---
params: {місяців: 12}
---
|> Гарантійний строк становить |(місяців) |(plural(місяців, "місяць", "місяці",
"місяців")).

|< Гарантія
|include("fragments/гарантія.f.prml", місяців=24)
```

- Шлях — від файлу з командою; розширення можна пропустити (`fragments/гарантія`).
- Імена, визначені у фрагменті, документові не видно; щоб передати значення назовні, фрагмент експортує його: `|export(строк=…)`, функцію — декоратором `@export` над `def`, готовий макрос — `|export("підпис")` (сторінка «Код»).
- Частину фрагмента позначають `|part("назва"):` … `|end part` і вставляють `|include("файл", part="назва")`.

6.4. Модулі: макроси й функції

Модуль `*.m.prml` містить лише визначення — макроси `|def`, функції й дані в `|code:`. Тексту в ньому немає.

```
|def підпис(сторона, особа):
|.SignatureRole: |(сторона)
|.SignatureLine: _____________ |(особа)
|enddef

|import("lib/підписи")
|підписи.підпис("Виконавець", виконавець.директор)
```

Після `|import("lib/підписи")` визначення доступні як `підписи.ім'я`; `|import("lib/підписи", "підпис")` — без префікса, `alias="п"` — під коротким іменем. Експортувати з модуля нічого не треба: відкриті всі імена, крім тих, що починаються з `_`. `|import` документа діє й у його фрагментах. Модуль обчислюється один раз на збирання.

6.4.1. Спільна бібліотека кількох проєктів

Шлях `|import` не виходить за корінь проєкту, але символічне посилання, створене всередині проєкту, може вести до спільного каталогу (рішення 136):

```
ln -s ../shared-lib lib     # у корені проєкту
```

Після цього `|import("lib/ціни")` бере модуль із `../shared-lib`. Збирання видає попередження P2639 з іменем посилання та його ціллю: каталог поза проєктом не потрапляє до архіву вихідних текстів, тому зберігайте його разом із проєктами (або в тому самому репозиторії). Шляхи всередині бібліотеки відраховуються від `lib/…`, і `../` у тексті, як і раніше, не виводить за корінь. У звіті й повідомленнях файл записується як `lib/ціни.m.prml`.

6.5. Дані з файлів

```
номер,дата,сума
1,2026-10-31,5400.00
```

```
2,2026-11-30,5400.00

|code:
акти = load_csv("data/акти.csv", types={"номер": "decimal", "дата": "datetime",
"сума": "money"})
|end
|for(a in акти):
|- Акт № |(a.номер) від |(fmt_date(a.дата, "short")) на |(money(a.сума, "UAH")).
|endfor
```

Без `types=` усі значення CSV — рядки (коди з провідними нулями не псуються). YAML і JSON читають `load_yaml` і `load_json`; відображення стають записами з доступом через крапку.

Таблиці LibreOffice Calc і Excel читають `load_ods` і `load_xlsx` (рішення 138): аркуш — `sheet="Позиції"` або номер з 1, діапазон — `range="A1:D20"`, перший рядок — імена полів (`header=False` — поля `A`, `B`, …), `types` — як у CSV.

```
|code:
позиції = load_ods("data/позиції.ods", sheet="Позиції", types={"ціна": "money"})
|end
```

Формули не обчислюються: береться результат, збережений у файлі, — збережіть таблицю в Calc або Excel після правки. Числа — `decimal` до 15 значущих цифр, дати — `datetime`; помилка в клітинці (`#DIV/0!`) або формула без результату зупиняють збирання з назвою аркуша й клітинки. Файли `.xls` спершу збережіть як `.xlsx` або `.ods`.

Увесь текстовий файл одним рядком повертає `load_text` (рішення 140): UTF-8, кінці рядків — LF, без розбору. Так зручно тримати поза документом SVG малюнка, знака чи схеми і вставляти його через `svg=`:

```
|img(svg=load_text("img/знак.svg"), width="12mm")
```

6.6. Збирання проєкту

```
primula build project.p.prml --pdf      # усі цілі проєкту
primula check project.p.prml            # перевірка всіх цілей без запису
primula build договір.prml              # один документ (з параметрами й стилями
проєкту)
primula check project.p.prml --dump-project   # опис проєкту в JSON
```

Цілі можна породжувати кодом: секція коду файлу проєкту (після заголовка) читає дані й викликає `build()` — наприклад, по акту на кожен рядок CSV:

```
---
kind: project
---
for row in load_csv("data/акти.csv", types={"номер": "decimal", "сума": "money"}):
    build("акт.prml", params={"номер": row.номер, "сума": row.сума}, output=f"out/акт-
{row.номер}.odt", pdf=True)
```

Помилка однієї цілі не зупиняє інших. Повний приклад із договором, актами з CSV, картою ключів і зведеним пакетом — `examples/07-project`.

6.7. Експорт у Markdown, HTML, DOCX

`primula export` записує обчислений документ в інші формати через pandoc: номери пунктів, посилання, виноски, таблиці й підписи зберігаються, розмітка сторінок (колонтитули, колонки, номери сторінок) — ні.

```
primula export договір.prml                    # договір.md
primula export договір.prml --to html          # один файл HTML із зображеннями
```

```
primula export договір.prml --to docx          # стилі абзаців — за іменами (custom-
style)
primula export договір.prml --to json          # AST pandoc, pandoc не потрібен
```

6.8. Швидкий перегляд: primula view

`primula view` збирає документ у тимчасовому каталозі (ODT і PDF, як `build --pdf`), відкриває PDF у переглядачі й видаляє файли, коли вікно закрито. Якщо збірка не вдалась, замість PDF показується журнал збірки — у терміналі або у вікні. Команда потрібна для файлових менеджерів: коротке ім'я `primula-view` лежить поруч із `primula`.

```
primula view договір.prml
primula view договір.prml --param номер=12 --viewer "okular --unique"
primula view --text договір.prml | less        # без LibreOffice: простий текст через
pandoc, близько секунди
```

Переглядач береться з `--viewer`, потім зі змінної `PRIMULA_VIEWER`, потім це переглядач PDF стільниці (okular, evince, atril, xreader, zathura, mupdf, qpdfview), потім `xdg-open`. Заданий і впізнаний переглядач очікується; `xdg-open` повертається одразу, тому файли залишаються в `$XDG_RUNTIME_DIR/primula-view` і видаляються пізніше. LibreOffice стартує з профілем у `~/.cache/primula`, що трохи прискорює повторні запуски; основний час забирає верстка в LibreOffice, тому для документа на кілька сторінок чекайте кілька секунд, а для швидкого погляду на текст використовуйте `--text`.

F3 у Midnight Commander — у `~/.config/mc/mc.ext.ini`:

```
[primula]
Extensions=prml
View=primula-view %f
```

Для тексту в самому терміналі: `View=%view{ascii} primula view --text %f`.

6.9. Зведений документ

Кілька документів збираються в один ODT командою `|document`:

```
|document("договір.prml", scope="договір")
|document("акт.prml", scope="акт1", params={"номер": 1})
```

Кожна частина починається з нової сторінки, нумерується у своїй області й обчислюється окремо; зміст і нумерація сторінок — спільні. Посилання на іншу частину — за повним ID: `|@("акт1:перелік")`.

6.10. Посилання на інший документ

Акт може посилатися на пункт договору, якщо договір названо в `ref_documents:` проєкту, а акт перелічує його в `references:` свого заголовка: `|@("оплата", "label")` в акті дасть «п. 2.2» договору. Текст посилання береться з реєстру договору, який транслятор оновлює під час збирання.

6.11. Карта ключів

Ключі `keys:` проєкту дають ресурсам і фрагментам імена: `|img(key="логотип")`, `|include(key="умови.оплата")`. Ціль збирання може підмінити ключ (`keys:` у цілі) — так один текст збирається з різними логотипами чи варіантами умов.

6.12. Проєкт замовлення

Для серії замовлень за одним договором роблять проєкт у підкаталозі з `extends: ../../project.p.prml`: він успадковує параметри, стилі, ключі й документи базового проєкту та додає власні дані замовлення. У Studio — «Файл → Створити → Проєкт замовлення».

7. Стилі

Оформлення документа задають стилі, а не текст: шрифт, відступи, інтервали, рамки таблиць, поля сторінок. У тексті пишеться лише *що* це — заголовок, пункт, реквізити, — а *як* це виглядає, вирішує модуль стилів. Тому той самий документ можна надрукувати інакше, підключивши інший набір, а правка оформлення не зачіпає тексту.

7.1. Вибір набору

У комплекті транслятора — набори стилів для типових документів:

Набір	Для чого	Основа
`std/styles/default`	підключається сам, якщо стилі не названо	—
`std/styles/business`	договори, акти, рахунки, протоколи; ДСТУ 4163:2020	`default`
`std/styles/letter`	службові листи; ДСТУ 4163:2020	`business`
`std/styles/technical`	настанови, інструкції, технічні описи	`default`
`std/styles/report`	звіти у сфері науки і техніки; ДСТУ 3008:2015	`default`
`std/styles/compact`	пам'ятки, довідки, шпаргалки	`default`

Документ називає набір одним рядком заголовка:

```
---
doc: {title: "Акт", language: uk-UA}
styles: [std/styles/business]
---
|.DocumentTitle: АКТ № 14
```

Основу (`default`) перелічувати не треба: набір сам її підключає. Стилі кожного набору, їхні властивості й типовий шрифт — у додатку «Стилі комплекту»; зразки — документи `examples/08-styles` (їх можна зібрати й переглянути PDF).

7.2. Застосування стилів у тексті

Запис	Що робить
`|.DatePlace: м. Харків`	стиль абзацу для абзацу цього рядка
`|.Note:` … `|end.Note`	стиль абзацу для кількох абзаців (оточення)
`|.DefinedTerm{Замовник}`	символьний стиль для фрагмента всередині рядка
`|*{…}`, `|/{…}`, `|`{…}`	напівжирний, курсив, моноширинний — це теж символьні стилі (`Strong`, `Emphasis`, `Code`)
`|table(style="TableCompact")`	стиль таблиці
`|list("DashList")`	стиль наступного списку

Ім'я стилю в `|.Ім'я` — ідентифікатор: воно не перекладається й не змінюється під час вичитування тексту. Невідоме ім'я — помилка з позицією.

Заголовки, пункти, списки, таблиці й підписи отримують стилі автоматично — їх задає прив'язка конструкцій у модулі стилів (`Heading1`…`Heading6`, `Clause`, `BulletList`, `TableGrid`, `TableCaption` …). Змінити стиль конструкції до кінця оточення можна командою `|set`:

```
|set(list="DashList")
|- перший пункт;
|- другий пункт.
```

Пряме форматування `|fmt(bold=True){…}` теж є, але дає попередження: разове оформлення губиться під час зміни набору, а однакові місця оформлюються по-різному. Якщо оформлення повторюється — заведіть стиль.

7.3. Власний модуль стилів

Не правте наборів комплекту — зробіть власний модуль над набором. Він містить лише відмінності й називає основу ключем `extends`:

```
---
kind: styles
extends: std/styles/business
defaults: {font: Roboto, size: 11pt}
paragraphs:
  Body:        {line_height: "120%"}
  Реквізити:   {parent: Details, keep_together: true}
characters:
  Акцент:      {bold: true, color: "#1F4E79"}
tables:
  TableServices: {header_background: "#DDE6F0"}
---

---
doc: {title: "Договір", language: uk-UA}
styles: [styles/компанія.s.prml]
---
Текст з |.Акцент{акцентом}.

|.Реквізити: Код ЄДРПОУ 00000000
```

- Документ (чи проєкт) називає лише власний модуль: основа та її основа (`business` → `default`) підключаються самі.
- Властивість стилю з тим самим іменем доповнює стиль набору: `Body: {line_height: "120%"}` змінює лише інтервал, решта лишається від набору.
- Новий стиль успадковує наявний через `parent`.
- Шрифт усього документа змінюється одним рядком `defaults: {font: …}`. Шрифт має бути встановлений там, де документ відкривають чи збирають PDF.

Групи стилів у модулі: `paragraphs` (стилі абзаців), `characters` (символьні), `tables`, `lists`, `blocks`, `frames` (розміри й розміщення зображень), `pages` (сторінки). Властивості — `font`, `size`, `bold`, `align`, `line_height`, `space_before`, `space_after`, `left`, `first_line`, `keep_with_next` та інші; повний перелік — розділ 11.3 специфікації. Підказки щодо властивостей і стилів дає Studio (наведення миші, `Ctrl+Space`).

7.4. Шари

Якщо стилі названо в кількох місцях, вони накладаються шарами, від раннього до пізнього:

1. модулі зі `styles:` проєкту;
2. модулі, передані під час збирання `--style ФАЙЛ`;
3. модулі зі `styles:` документа;
4. стилі, визначені кодом самого документа.

Пізніший шар доповнює й перевизначає ранні. Основа модуля (`extends`) — шар одразу перед ним. Які шари справді підключено й звідки взявся кожен стиль, показує панель «Стилі» Studio або `primula check документ.prml --dump-styles`.

7.5. Сторінки й колонтитули

Поля й розмір сторінки — стиль сторінки (`pages:` модуля, типово `Portrait`; у наборах є `Landscape`). Колонтитули задаються в коді модуля стилів або в документі:

```
|footer("Portrait"):
|.Footer: Договір № П-2026/017 — сторінка |field.page з |field.pages
|end footer
```

Зміна сторінки з цього місця — `|page(style="Landscape")`, розрив — `|newpage`.

7.6. Перехід з ODT Markdown Kit

Діловий набір Kit 1.2 (`business-styles.yaml`) перенесено в `std/styles/business` і `default` під тими самими іменами, крім наведених нижче:

Kit	Primula
`TocTitle`, `TocEntry`	`ContentsTitle`, `Contents1`…`Contents6`
`StrongText`, `EmphasisText`, `UnderlinedText`	`Strong`, `Emphasis`, `Underline` (короткі форми `|*{}`, `|/{}`, `|_{}`)
`DraftPlaceholder`	`Placeholder` (коротка форма `|?{…}`)
`BodyRoboto`, `FontCarlito` і подібні	немає: шрифт задається `defaults: {font: …}` власного модуля
`Recipient`, `LetterSubject`, `Salutation`	у наборі `letter`
`Approval`	у `business`, на 100 мм від лівого поля (ДСТУ 4163:2020)
`NumberedList` з літерами `а)`	`NumberedList` (`1.`, `1.`, `а)`), `LetterList` — літери мови документа «а)», `LatinLetterList` — «a)»
пресети зображень `Logo`, `Illustration` …	стилі фреймів з тими самими іменами: `|img(…, style="Logo")`

Номери пунктів і посилання в Kit писалися текстом; у Primula їх ставить транслятор (`|>`, `|@("id")`), а стиль `Clause` застосовується до пунктів сам.

8. Повідомлення транслятора

Транслятор перевіряє документ цілком — від кодування до посилань і таблиць — і повідомляє про все, що знайшов, а не лише про першу помилку. Документ з помилками не збирається; з попередженнями — збирається, але попередження майже завжди вказує на справжню проблему.

8.1. Як читати повідомлення

```
договір.prml:6:9: error[P2201]: ім'я 'сумма' не визначено
 6 | Сума: |(сумма).
   |         ^^^^^
договір.prml: 1 помилка, 0 попереджень
```

- `договір.prml:6:9` — файл, рядок, стовпець. Помилка у фрагменті чи модулі вказує їхній файл і місце підключення.
- `error` — помилка, `warning` — попередження.
- `P2201` — код. Перша цифра — область: P0 командний рядок і файли, P1 синтаксис, P2 імена й обчислення, P3 стилі, P4 структура й посилання, P5 таблиці, P6 сторінки й графіка, P7 модулі й проєкт, P8 запис ODT, P9 технічна документація. Повний перелік із текстами — додаток «Повідомлення».
- Рядок із `^^^` показує місце; рядки `= help:` і `= note:` — підказка й пояснення.

Мова повідомлень — `--lang uk|ru|en` або змінна `PRIMULA_LANG`. Для програм і редакторів — `--format json`: список повідомлень із полями `code`, `severity`, `message`, `file`, `line`, `column`. Код завершення: 0 — без помилок, 2 — є помилки.

У Studio повідомлення перевірки зібрано в панелі; клацання відкриває місце в редакторі, F1 на конструкції — її розділ специфікації.

8.2. Попередження і випуск

`--strict` перетворює попередження на помилки з тим самим кодом. Так збирають остаточну редакцію: незаповнені місця `|?{…}`, неоформлені числа й пряме форматування не пройдуть.

```
primula build договір.prml --strict --pdf
```

8.3. Часті помилки

8.3.1. Стиль без тексту — P1209

Ім'я стилю після `|.` — ідентифікатор, а не текст. Якщо під час правки воно змінилося («Місце-дата» → «Місце і дата»), рядок перестає бути стилем:

```
|.Місце-дата
```

Поверніть ім'я стилю й пишіть текст після двокрапки: `|.Місце-дата: м. Харків, 12.10.2026`. Список імен стилів — панель «Стилі» Studio або `--dump-styles`.

8.3.2. Ім'я не визначено — P2201

```
Сума: |(сумма).
```

Помилка в імені параметра чи змінної, параметр не оголошено в `params:` документа чи проєкту, функція модуля без `|import`. Модуль `std/math` у документі підключають явно: `|import("std/math")`.

8.3.3. Команда «з'їла» слово — P2601

```
|(1)|sellerom
```

Літера одразу після імені команди продовжує ім'я: `|sellerom` — це команда `sellerom`. Роздільник `|;` закінчує ім'я: `|seller|;ом`.

8.3.4. Пробіл після риски — P1201

```
Ціна | 100 грн.
```

Після `|` мають іти літера, `(` або знак короткої форми. Сама риска в тексті пишеться двома: `||`.

8.3.5. Неправильне закриття — P1218, P1219, P1224

`|end` закриває найближче оточення, але не керівні конструкції: `|if` закривається `|endif`, `|for` — `|endfor`, `|def` — `|enddef`. Блок `|code:` закривається рядком з одного `|end`, без імені (`|end code` — помилка P1224). Пункт у рядковій формі (`|> текст`) оточенням не є, і `|end` після нього — зайвий (P1218).

```
|> Пункт.
|end
```

8.3.6. Значення не можна вивести — P2603, P2604

`|(…)` виводить рядок, число, дату або розмітку. `None`, логічне значення, список чи запис — помилка P2603: використовуйте `str()`, `join_list()` або поле запису.

Дробове число, сума чи дата без функції оформлення — попередження P2604:

```
|(dec("1.5"))
```

Оформлюйте бібліотекою: `num(x, 2)`, `money(x, "UAH")`, `fmt_date(d, "long")`.

8.3.7. Невідомий стиль — P3108

```
|.Невідомий: текст
```

Стилю немає в жодному підключеному модулі: помилка в імені або не підключено набір (`styles:`). Які модулі підключено — `--dump-styles` або панель «Стилі».

8.3.8. Посилання й ID — P4301, P4102

```
Див. |@("оплата").
```

Цілі з таким `id=` немає: помилка в імені, ціль вилучено чи виключено умовою. Повторний `id=` — помилка P4102 з позиціями обох місць.

8.3.9. Пропуск рівня — P4201

```
|< Розділ
|<<< Підрозділ
```

Заголовки йдуть за рівнями: після `|<` — `|<<`, а не `|<<<`. Так само зі списками й пунктами.

8.3.10. Таблиця — P5103, P1226

```
|table(cols=["*", "*"]):
|: а |& б |& в
|end table
```

Кількість клітинок рядка не збігається з кількістю стовпців (з урахуванням `span=`). Порожня клітинка пишеться явно: `|&` без тексту. Текст усередині таблиці поза рядком `|:` — помилка P1226.

8.3.11. Ключ заголовка перейменовано — P1120

Ключі `document:` і `documents:` попередніх версій перейменовано: метадані — `doc:`, у проєкті — `doc_defaults:` і `ref_documents:`.

8.3.12. Заповнювач — P2635

`|?{…}` залишено в документі. Під час чорнового збирання це попередження, з `--strict` — помилка.

8.4. Якщо незрозуміло, що не так

- Перевірте ділянку окремо: `primula console документ.prml` відкриває консоль обчислювача з іменами документа — там можна обчислити вираз (`:help ім'я` — опис функції).
- `primula check документ.prml --dump-tree` показує, як транслятор зрозумів структуру: де закінчується пункт, що ввійшло до таблиці.
- У Studio права кнопка миші на конструкції дає підказку й розділ специфікації.

9. Паралельні тексти

Оточення `|parallel` ставить поруч два або більше текстів — потоки — і вирівнює по горизонталі початки їхніх смислових блоків — сегментів. Так пишуться двомовний договір, протокол розбіжностей («було / стало»), текст із коментарем на полях. Колонки розділу (`|section(cols=2)`) для цього не годяться: вони течуть незалежно. Таблиця годиться на вигляд, але в ній немає поняття «пункт і його переклад»: нумерація, посилання й мова спільні на всю таблицю.

Транслятор виводить `|parallel` таблицею без рамок, але знає потоки: кожен потік нумерується своїми лічильниками, обчислюється своєю мовою і звіряється з основним потоком.

9.1. Приклад

```
---
doc: {title: "Договір / Agreement", identifier: "ДП-2026/017", language: uk-UA}
params:
  ціна: 48000.00
  виконавець: {uk: "ТОВ «Виконавець»", en: "Vykonavets LLC"}
styles: [std/styles/business]
---
|parallel(tracks=[track("uk", lang="uk", title="Українською мовою"), track("en",
lang="en", title="In English")],
          gap="8mm", rule="0.3pt solid #999999", parity="translation"):

|segment(span="all"):
|.DocumentTitle: ДОГОВІР № |(doc.identifier) / AGREEMENT No. |(doc.identifier)
|end segment

|segment(id="subject"):
|track("uk"):
|<(id="subject") Предмет договору
|>(id="services") |(getattr(виконавець, track.id)) надає послуги з розроблення
документації.
|> Ціна послуг — |(money_full(ціна, "UAH")) (див. |@("services", "label")).
|track("en"):
|<(id="subject") Subject of the Agreement
|>(id="services") |(getattr(виконавець, track.id)) provides the services of developing
documentation.
|> The price of the services is |(money_full(ціна, "UAH")) (see |@("services",
"label")).
|end segment

|end parallel
```

В українській колонці сума виходить «сорок вісім тисяч гривень 00 копійок», посилання — «п. 1.1»; в англійській — «forty-eight thousand hryvnias 00 kopiykas» і «cl. 1.1». Пункти нумеруються в обох колонках однаково.

9.2. Потоки

Потік описується функцією `track(id, …)` в аргументі `tracks=`:

Аргумент	Що задає
`lang`	мова потоку: суми прописом, дати, лапки, мітки посилань, перевірка правопису у Writer
`title`	назва над колонкою; рядок назв повторюється на кожній сторінці
`width`	ширина колонки, як у стовпця таблиці: `"30%"`, `"60mm"`, `"*"`

Аргумент	Що задає
`primary=True`	основний потік (типово перший): від нього йде нумерація документа, зміст, навігація, з ним звіряються інші
`numbered=False`	потік без нумерації: коментарі, примітки на полях
`optional=True`	потік може не мати блоку в сегменті — клітинка лишається порожньою
`block`	стиль блоку для всіх клітинок потоку (тло, відступи)

Якщо потрібні лише мови, вистачає скорочення `langs=["uk", "en"]`. Проміжок між колонками — `gap=`, лінія — `rule=`.

Параметри документа спільні для всіх потоків. Значення мовою потоку вибирається кодом: у блоці потоку доступний запис `track` (`track.id`, `track.lang`), тому `|(getattr(виконавець, track.id))` бере поле `uk` або `en`.

9.3. Сегменти

Сегмент — рядок вирівнювання: блок кожного потоку починається на одному рівні, кінець — за найдовшим блоком. Сегментом зазвичай роблять розділ договору або пункт.

- **Оточення:** `|segment(id="…"):`, далі блоки потоків `|track("uk"):`, `|track("en"):` і `|end segment`. Блок потоку закінчується на наступному `|track` або на закритті сегмента.
- **Рядок** — для коротких блоків:

```
|parallel(langs=["uk", "en"]):
|segment(id="city") |track("uk"){м. Харків} |track("en"){Kharkiv}
|end parallel
```

- **На всю ширину** — спільний текст для всіх потоків (назва договору, таблиця підписів): `|segment(span="all"):` без блоків потоків.

Усередині блоку потоку пишеться звичайний текст: заголовки, пункти, списки, таблиці, рисунки, виноски. Не можна: зміна сторінки, розділи з колонками, додатки, колонтитули, зміст, вкладений `|parallel` (повідомлення P5304). Між сегментами допустимі `|newpage` і `|page(…)`: таблиця продовжується на новій сторінці з рядком назв.

Заголовок, що лишився останнім на сторінці, Writer не переносить до наступного сегмента. Пишіть заголовок в одному сегменті з першим пунктом розділу.

9.4. Нумерація, посилання, виноски

- Кожен потік має свої лічильники: на вході в `|parallel` вони копіюються з документа, після оточення документ продовжує нумерацію основного потоку.
- ID усередині потоку отримує ім'я потоку: `id="services"` в українському блоці — це `uk.services`, в англійському — `en.services`. Тому однакові `id=` в обох потоках не конфліктують.
- `|@("services")` усередині потоку шукає ціль у своєму потоці, далі в основному, далі в усьому документі. Ціль іншого потоку пишеться повністю: `|@("uk.services")`.
- Виноска неосновного потоку отримує номер свого потоку: в обох колонках номери однакові.
- Заголовки неосновних потоків виглядають як заголовки, але в навігатор Writer і закладки PDF не потрапляють — там лише основний потік.

Зміст документа бере заголовки основного потоку. Зміст іншого потоку — `|toc(track="en")`; типова назва — мовою потоку («CONTENTS»):

```
|toc(title="ЗМІСТ")
|toc(track="en")
|parallel(langs=["uk", "en"]):
|segment:
|track("uk"):
|< Предмет договору
|track("en"):
|< Subject of the Agreement
|end segment
|end parallel
```

9.5. Перевірка відповідності

Аргумент `parity=` звіряє кожен нумерований потік з основним у кожному сегменті:

Профіль	Що перевіряється
`structure` (типово)	чи є блок кожного обов'язкового потоку; послідовність заголовків, пунктів, елементів списків, таблиць, рисунків, приміток
`translation`	те саме, а також ID, цілі посилань, імена параметрів у `|(…)`, числа й адреси в тексті, лічильники потоків наприкінці
`none`	нічого

Сегмент може задати свій профіль: `|segment(id="idiom", parity="none")` — для вільного перекладу. Розбіжності — попередження P5310–P5315; з `--strict` вони стають помилками.

```
|parallel(langs=["uk", "en"], parity="translation"):
|segment(id="term") |track("uk"){Строк — 5 днів.} |track("en"){The term is 7 days.}
|end parallel
```

9.6. Переклад: заготовки, підтвердження, актуальність

Коли документ перекладається, зручно вести стан кожного блоку:

1. **Заготовки.** `primula parallel договір.prml --stub --track en` дописує в сегменти без англійського блоку копію структури основного блоку із заповнювачами `|?{…}`. Заповнювачі — попередження, з `--strict` — помилки, тому незаповнений переклад не потрапить у випуск.
2. **Підтвердження.** Перевірений переклад фіксується: `primula parallel договір.prml --stamp --track en --status reviewed` записує в блоки `status="reviewed"` і `rev="…/…"` — хеші тексту основного блоку і самого перекладу. `--segment ID` підтверджує лише названі сегменти.
3. **Зміна оригіналу.** Якщо після підтвердження змінено український текст, `check` видає P5320 «переклад застарів»; якщо змінено сам переклад — P5321. Попередження лишається, доки переклад не виправлено й не підтверджено знову.
4. **Перед випуском.** `primula parallel договір.prml --check-status reviewed` завершується помилкою, якщо якийсь блок не в цьому стані або застарів.

У Studio ті самі дії — у меню «Збирання → Паралельні потоки» і в контекстному меню редактора («Підтвердити переклад сегмента …» для блоку під курсором). Панель «Структура» показує сегменти й блоки потоків зі станом і позначає застарілі.

Хеш рахується за текстом блоку без коментарів і зайвих пробілів, тому переформатування не робить переклад застарілим. Вирази `|(…)` входять у хеш як записані: зі зміною значення параметра переклад не застаріває — значення спільні для потоків.

9.7. Протокол розбіжностей

Три потоки: редакція постачальника (необов'язкова — нового пункту в ній немає), редакція покупця (основний потік) і коментар юриста (без нумерації, вузька колонка). Номери пунктів договору пишуться явно:

```
|parallel(tracks=[track("old", title="Редакція Постачальника", optional=True),
                  track("new", title="Редакція Покупця", primary=True),
                  track("note", title="Коментар", width="30%", numbered=False,
optional=True)]):
|segment(id="p-4-2"):
|track("old"):
|*{4.2.} Оплата здійснюється протягом 5 банківських днів.
|track("new"):
|*{4.2.} Оплата здійснюється протягом 30 календарних днів.
|track("note"):
|.SmallPrint: Відстрочку погоджено листом від 12.10.2026.
|end segment
|segment(id="p-4-3"):
|track("new"):
|*{4.3.} Покупець має право на дострокову оплату.
|end segment
|end parallel
```

Повні приклади — `examples/09-parallel/agreement.prml` (двомовний договір з підтвердженими перекладами) і `protocol.prml`.

9.8. Стилі

Таблиця виводу — стиль `ParallelTable` (без рамок), рядок назв потоків — `ParallelHeader`; обидва є в усіх наборах комплекту (додаток стилі комплекту). Свої імена задаються в модулі стилів: `structure_styles(parallel_table="…", parallel_header="…")` або аргументом `style=` у `|parallel`.

10. Код: діалект Python

Код Primula пишеться як Python: вирази `|(…)`, аргументи команд, блоки `|code:`, функції й макроси. Синтаксис — підмножина Python 3.11, але зміст подекуди інший: числа десяткові, глобальні імена незмінні, виведення (`print`) немає. Ця сторінка перелічує те, що треба знати авторові, який знає Python; що працює як у Python, тут не повторюється.

Повний приклад із таблицями «вираз → Primula → Python 3» — `examples/10-code` (документ `code.prml`, фрагменти за темами, файли помилок `forbidden.prml` і `mistakes.prml`). Приклад показує вихідний текст: код і визначення в PDF не друкуються, тому PDF читається разом із вихідним текстом. Функції бібліотеки — у додатку «Функції виразів».

10.1. Де пишеться код

Запис	Призначення
`|(сума * 2)`	вивести значення в текст
`|table(cols=["*", "30mm"])`	аргументи команд — вирази
`|if(n == 1){день}|else{дні}`, `|for(p in позиції){…}`	умова й цикл; тіло — розмітка, а не код
`|code:` … `|end`	визначення, функції, розрахунки
`|let(n=5)`	визначення посеред тексту
`|def підпис(роль):` … `|enddef`	макрос: функція, тіло якої — розмітка

10.2. Числа: decimal замість int і float

Ціле число й число з крапкою без експоненти — `decimal`: точне десяткове число до 64 значущих цифр, яке пам'ятає кількість знаків після коми. Окремого `int` немає: `type(5)` — `"decimal"`. Двійковий `float` виходить лише з літерала з експонентою (`1.5e3`) або `float(x)`; `decimal` разом із `float` дає `float`.

Вираз	Primula	Python 3
`0.1 + 0.2 == 0.3`	`True`	`False`
`2.20 * 3`	`6.60`	`6.6000000000000005`
`7 / 2`	`3.5` (`decimal`)	`3.5` (`float`)
`round(2.5)`	`3`	`2`
`round(2.675e0, 2)`	`2.68`	`2.67`
`10 ** 70`	помилка P2309	ціле з 71 цифри
`True + 1`	помилка P2304	`2`

Округлення одне для всіх чисел — половина від нуля, `float` округлюється за своїм десятковим записом. Гроші — лише `decimal`: грошові функції не приймають `float`.

```
Ціна: |(money(2.20 * 3, "UAH")), разом із ПДВ |(money(round(2.20 * 3 * 1.2, 2), "UAH")).
```

10.3. Рядки

Рядок — символи Unicode, як у Python 3: `len("Київ") == 4`. Відмінності:

- є лише методи зі списку специфікації (розділ 12.10): `split`, `join`, `strip`, `replace`, `startswith`, `find`, `upper`, `lower`, `title`, `zfill`, `format` і деякі інші; `center`, `partition`, `encode` — помилка P2313;
- f-рядки — за синтаксисом Python 3.11: усередині фігурних дужок не можна лапок того самого виду, що навколо рядка (P1208); пишіть `f"{d['k']}"`;
- `"{0.name}".format(r)` — помилка P2330; поле запису читається за ключем: `"{0[name]}"`;
- специфікація формату (`f"{x:,.2f}"`) — англійська й мови документа не знає; для тексту є `num()`, `money()`, `fmt_date()`.

```
|code:
сума = 1234.5
|end
Python-формат: |(f"{сума:,.2f}"); формат документа: |(num(сума, 2)).
```

10.4. Списки, кортежі, словники

Як у Python, разом із генераторами списків і словників та розпакуванням. Відмінності:

- множин немає: `{1, 2}` — помилка P2101; унікальність — словником або `not in`;
- операторів `|`, `&`, `^`, `<<`, `~` немає; словники об'єднують `{**a, **b}`;
- `is` — лише з `None` (P2103);
- ключ словника — незмінне значення; `float` і довжина ключем бути не можуть (P2317); `5` і `5.00` — один ключ;
- `range()` і `reversed()` повертають список;
- `lambda` немає: функція для `key=` — звичайний `def`.

```
|code:
def мінус(x):
    return -x
порядок = sorted([3, 1, 2], key=мінус)
|end
|(join_list([str(x) for x in порядок]))
```

10.5. Записи

Запис (`record`) — незмінне значення з полями: `r.назва` або `r["назва"]`. Записами стають параметри заголовка YAML, дані проєкту, результати `load_csv`, `load_yaml`, `load_json`. У Python схожий `namedtuple`, але запис — не послідовність:

- `"назва" in r` і `len(r)` — помилки; поле перевіряють `hasattr(r, "назва")`;
- відсутнє поле — помилка P2314; `getattr(r, "ціна", 0)` дає значення за замовчуванням;
- новий запис зі зміненим полем — `r.replace(кількість=4)`; у словник — `dict(r)`.

```
---
doc: {language: uk-UA}
params:
  сторона: {назва: "ТОВ «Альфа»", код: "12345678"}
---
|(сторона.назва), ІПН: |(getattr(сторона, "ІПН", None) or "—").
```

YAML читається за правилами Primula: `15.00` — `decimal` із двома знаками, дата без лапок — `datetime`.

10.6. Дати й довжини

Дата й момент — один вбудований тип `datetime`, без `import`; точність — секунда, поясів немає.

- `дата + 30`, `дата - 30` — зсув на доби; `дата - дата` — число діб (`decimal`), а не `timedelta`;
- `hours(36)`, `weeks(2)` — числа діб: `дата + hours(36)`; календарні місяці — `add_months(дата, 1)` (31 січня + 1 місяць = 28 лютого);
- `weekday` — атрибут від 1 (понеділок) до 7, а не метод від 0;
- у текст дата виводиться `fmt_date(дата, "contract")`.

Довжина — число з одиницею: `10*mm` або `"10mm"`; константи `mm`, `cm`, `pt`, `inch` (не `in`). Результат — в одиниці лівого операнда, порівняння — з допуском 0,001 мм.

```
|code:
підписано = date(2026, 10, 12)
оплата = підписано + 10
|end
Оплата до |(fmt_date(оплата, "contract")) — через |(оплата - підписано) днів, |
(fmt_date(add_months(підписано, 1), "long")) — через місяць.
```

10.7. Функції

`def` зі значеннями за замовчуванням, `*args`, `**kwargs`, параметрами лише за іменем, рекурсією, вкладеними функціями й замиканнями; усередині функції змінні змінювані, є `while`. Відмінності:

- немає `lambda`, генераторів (`yield`), `global`, `nonlocal`; декоратор єдиний — `@export` (розділ «Модулі й фрагменти»);
- значення за замовчуванням обчислюються один раз і заморожуються: пастка Python `def f(x, acc=[])` тут — помилка P2318 під час першого `append`;
- глобальний список, переданий у функцію, теж заморожений: будується новий список;
- замість межі рекурсії Python — ліміти: глибина 500, 20 с на одне обчислення, 10 000 000 елементів.

```
---
doc: {language: uk-UA}
params:
  позиції:
    - {назва: "Папір", кількість: 20, ціна: 120.50}
    - {назва: "Ручки", кількість: 100, ціна: 15.00}
---
|code:
def рядки(позиції):
    result = []
    for i, p in enumerate(позиції, 1):
        result.append(record(n=i, назва=p.назва, сума=p.кількість * p.ціна))
    return result
|end
|table(cols=["10mm", "*", "35mm"]):
|: № |& Найменування |& Сума, грн
|for(p in рядки(позиції)):
```

```
|: |(p.n) |& |(p.назва) |& |(num(p.сума, 2))
|endfor
|end table
```

10.8. Глобальні імена: ліниві й незмінні

Це друга велика відмінність після чисел. Python виконує модуль згори донизу й дозволяє переприсвоїти ім'я. У Primula визначення верхнього рівня — `x = …`, `def`, `|let`, параметри заголовка — збираються з усього файла заздалегідь, обчислюються під час першого звернення й заморожуються.

- Порядок у тексті неважливий: ім'я можна використати вище за його визначення.
- Повторне визначення — помилка P2203, `x += 1` на верхньому рівні — P2105, цикл визначень — P2204. Параметр заголовка теж не можна перевизначити в коді.
- Визначення, яке ніхто не читає, не обчислюється — і його помилка не проявиться, доки ім'я не знадобиться.
- Решта операторів верхнього рівня (`for`, `if`, виклики) — дії: виконуються після збирання визначень, а імена всередині них локальні (попередження P2501).
- Умовне значення — умовний вираз: `ставка = 20 if платник_пдв else 0`.
- Накопичення — генератор списку або функція, а не `append` у циклі верхнього рівня (P2318).

```
|code:
разом = ціна * кількість
ціна = 250.00
кількість = 4
ставка = 20 if разом > 500 else 0
|end
Разом |(num(разом, 2)), ПДВ |(ставка) %.

|code:
x = 1
x = 2
|end
```

10.9. Модулі й фрагменти

Модуль (`.m.prml`, `|import`) містить лише визначення. Усі його імена без `_` на початку відкриті, експортувати нічого не треба; ім'я з `_` закрите насправді (`модуль._имя` — помилка).

```
|code:
_ставка = 20

def з_пдв(сума, ставка=_ставка):
    return round(сума * (100 + ставка) / 100, 2)
|end
```

Підключення	Звернення
`|import("lib/ціни")`	`ціни.з_пдв(…)`, макрос — `|ціни.ім'я(…)`
`|import("lib/ціни", "з_пдв")`	`з_пдв(…)` без префікса
`|import("lib/ціни", alias="ц")`	`ц.з_пдв(…)` (as — ключове слово Python)

`|import` документа діє й у фрагментах, які документ підключає; `|import` усередині фрагмента — лише в цьому фрагменті. Шлях пишеться від файла з командою.

```
|import("lib/ціни")
Разом із ПДВ: |(num(ціни.з_пдв(1000.00), 2)) грн.
```

Фрагмент (`.f.prml`, `|include`) — текст документа з власними визначеннями, і вони закриті: один фрагмент можна підключити двічі без конфлікту. Документ бачить лише експортоване (рішення 135):

Запис у фрагменті	Що експортується
`@export` над `def f(…)` верхнього рівня `|code:`	функція під своїм ім'ям
`@export(name="g")` над `def _f(…)`	функція під ім'ям g; `_f` лишається закритою
`|export("підпис", "f")`	уже визначені імена: макроси `|def`, функції, `|let`
`|let(x=…, export=True)`, `|export(x=…)`	нове значення

```
---
kind: fragment
---
|code:
@export
def знижка(сума, відсоток):
    return round(сума * (100 - відсоток) / 100, 2)
|end
|def рядок(назва, сума):
|(назва) — |(num(сума, 2)) грн
|enddef
|export("рядок")

|include("fragments/знижки")
|рядок("Папір зі знижкою", знижка(2410.00, 5))
```

Якщо два фрагменти експортують одне ім'я, конфлікт розв'язується під час підключення: `|include(…, ns="f")` кладе експорт у `f.*`, `rename={…}` перейменовує. `@export` поза фрагментом — помилка P2610, над вкладеною функцією — P2106, у модулі — попередження P2638 (там усе й так відкрите). Функціям, якими користуються кілька документів, місце в модулі; із фрагмента експортують те, що стосується його тексту.

10.10. Значення в тексті й макроси

Виведення немає: значення потрапляє в документ лише виразом `|(…)`, і тип вирішує, як саме.

- Рядок, число, дата, розмітка (`content`) вставляються. Дробове число чи дата без оформлення — попередження P2604: оформлюйте `num()`, `money()`, `fmt_date()`.
- `None`, `bool`, список, словник, запис — помилка P2603, а не «None» чи «[1, 2]», як дав би `str()`. Необов'язкове значення — `|(x or "—")`, список — `join_list()`.
- Розмітка непрозора: `markup("…") + "…"` — помилка; рядки спершу з'єднують, потім викликають `markup()`.
- Функція з `|code:`, викликана як команда `|f(…)`, вставляє свій результат; макрос `|def` отримує тіло виклику параметром `body`.

```
|(None)

|code:
def жирно(текст):
    return markup("|*{" + текст + "}")
|end
|def позначка(текст, body):
|.Note: |*{|(текст):} |(body)
|enddef
|позначка("Увага"):
Текст із |жирно("виділенням").
|end
```

10.11. Чого в ядрі немає

Документ має збиратися однаково на будь-якій машині, тому в ядрі немає вводу-виводу (крім `load_*`), годинника (`build_date()` фіксується `--build-date`), випадкових чисел і доступу до внутрішніх об'єктів Python. Заборонено `import` (модулі підключаються `|import` у розмітці), `class` (дані — `record`), `try`/`raise` (помилка зупиняє збирання з позицією; `fail("…")` — навмисна помилка), `lambda`, `yield`, `async`, `global`, `nonlocal`, `del`, `with`, множини, побітові оператори, анотації, `else` у циклах, декоратори, крім `@export`, імена на кшталт `__class__`. Усе це виявляється ще під час розбору коду, до обчислення; повний перелік із замінами — у прикладі (`forbidden.prml`) і в розділі 12.2 специфікації.

```
|code:
квадрат = lambda x: x * x
|end
```

11. Бібліотеки: розробка й підключення

Бібліотека Primula — каталог із модулями `.m.prml` та їхніми даними, яким користуються кілька проєктів: перевірка реквізитів, імена у відмінках, графіки платежів, QR платежу. Її розробляють, перевіряють і випускають окремо від транслятора й від документів: бібліотека — звичайний текст Primula, вона не змінює мову й не потребує збирання транслятора.

Готовий зразок — `examples/11-library`: бібліотека `shared/ua-iban` (IBAN UA: перевірка, групи по чотири знаки, побудова) і проєкт `project/`, який підключає її символічним посиланням.

11.1. Бібліотеки комплекту

У репозиторії та в пакеті встановлення (`share/primula-docs/libraries/`) лежать готові бібліотеки версії 0.1.0. Кожна — каталог зі своїм `README.md` (функції, джерела правил, дата звірки, межі), тестами та прикладом `demo/`.

Бібліотека	Що робить	Основні функції
`ua_ids`	перевірка й оформлення ЄДРПОУ, РНОКПП, IBAN UA	`edrpou`, `rnokpp`, `iban_grouped`, `iban_make`, `party_code`
`names`	ПІБ у відмінках, ініціали, підпис, посади	`case`, `initials`, `signature`, `position_case`
`ua_data`	ставки НБУ та ПДВ, мінімальна зарплата, прожитковий мінімум, ІСЦ, календар — з датою чинності	`discount_rate_on`, `vat_rate_on`, `min_wage_on`, `cpi_on`, `calendar`
`legal`	пеня, 3 % річних, інфляційні втрати, строки	`penalty`, `claim_table`, `annual3`, `inflation_losses`, `deadline`
`finance`	ануїтет, графік платежів, NPV та IRR, лінійна амортизація	`annuity_payment`, `schedule`, `npv`, `irr`, `straight_line`
`estimate`	кошторис: підсумки за розділами, знижка, ПДВ, сума прописом; команди `|estimate.table(…)`, `|estimate.total_words(…)`	`table`, `total_words`, `totals`, `section_total`
`bib`	бібліографія за ДСТУ 8302:2015: нумеровані посилання `[3, с. 45]` і перелік літератури; команди `|bib.cite(…)`, `|bib.references(…)`	`registry`, `cite`, `references`, `entry`
`payment_qr`	QR платежу за правилами НБУ (гривня)	`payload`, `svg`, `link`
`epc_qr`	QR платежу SEPA (євро), EPC069-12	`payload`, `svg`
`codes`	контрольні цифри EAN-13, ISBN, Луна; штрихкод EAN-13	`ean13_check`, `isbn13`, `luhn_check`, `ean13_svg`
`charts`	стовпці, лінія й круг для `|figure(svg=…)`	`bar`, `line`, `pie`
`stats`	медіана, мода, дисперсія, перцентиль, ковзне середнє	`median`, `mode`, `variance`, `percentile`
`electronics`	ряди E3–E96, дільник, з'єднання, потужність, допуск	`series`, `nearest`, `divider`, `parallel`
`vcs`	таблиця змін технічного документа з історії Git	`changes`

Одна бібліотека підключається посиланням `lib` на її каталог і імпортується як `lib/<ім'я>` (так написано в її `README.md`); якщо бібліотек кілька, у кожної своє посилання `lib/<ім'я>` та імпорт `lib/<ім'я>/<ім'я>` (див. «Підключення до проєкту»):

```
ln -s ~/.local/share/primula-docs/libraries/ua_ids lib/ua_ids

|import("lib/ua_ids/ua_ids")
ЄДРПОУ |(ua_ids.edrpou(код)), IBAN |(ua_ids.iban_grouped(рахунок)).
```

Що потрібно знати:

- Модуль не може підключити сусідню бібліотеку (`../` заборонено), тому `legal` отримує дані аргументом: `legal.penalty(борг, початок, сплата, ua_data.series("discount"))`.
- `ua_data` зберігає значення з датою початку й джерелом. Дата пізніша за останній запис ряду — помилка, а не мовчазне «останнє значення»: дані треба оновлювати (новий рядок із новою датою, старі не змінюються). Дані бібліотеки звірено на 31.10.2026, календар — 2024–2026.
- `legal` рахує за формулами й даними бібліотеки; це не юридична консультація. Правові підстави розрахунку та їхні джерела перелічено в її `README.md` (закони звірено з текстами на zakon.rada.gov.ua; позиції судів — за повідомленнями Верховного Суду та оглядами, без повних текстів постанов): перед застосуванням у претензії перевірте їх.
- Функції із суфіксом `_problem` повертають причину текстом (порожній рядок, якщо все гаразд) і нічого не зупиняють; функції без суфікса при помилці зупиняють збірку.

11.2. Що може бібліотека

Модуль містить лише визначення: функції й константи в `|code:`, макроси `|def`, `|import` інших модулів (розділ «Модулі: макроси й функції»). Йому доступне все ядро — діалект Python (сторінка «Код: діалект Python») — і функції стандартної бібліотеки: числа й гроші, слова й відмінки, дати й робочі дні, `load_csv`, `load_yaml`, `load_json`, зображення з рядка SVG (`img(svg=…)`).

Чого в модуля немає і що робити:

Немає	**Натомість**
коду Python, `import`, вводу-виводу, мережі, годинника	дані — файли поруч із модулем (`load_yaml("data/…")`), дата — аргументом функції
`doc.*` документа (модуль обчислюється один раз на збирання)	мову й дату документ передає аргументом (`lang="uk"`); функції стандартної бібліотеки всередині модуля самі дотримуються мови документа
`ord()`, `chr()`, побітових операцій	позиція в рядку-абетці: `"АБС…".find(ch)`; таблиці значень
`assert`	`fail("…")` — зупиняє збирання з повідомленням і місцем виклику
важких обчислень	цикл ядра виконує близько десятків тисяч кроків за секунду, ліміт — 20 с на обчислення; алгоритм на сотні тисяч кроків (QR-код, геш) просять як функцію транслятора, а бібліотека будує поверх неї

11.3. Будова каталогу

```
ua-iban/                  бібліотека (свій репозиторій або каталог)
  README.md               призначення, версія, ліцензія, функції, джерела правил
  project.p.prml          корінь для тестів: tests/ імпортує ../iban
  iban.m.prml             публічний модуль: ім'я файла — ім'я простору (iban.*)
  _tables.m.prml          допоміжний модуль (необов'язково)
  data/                   дані модуля: YAML, CSV, JSON
  tests/iban-test.prml    документи-тести
  run-tests.sh            primula check для кожного тесту
```

- Імена без `_` на початку — інтерфейс бібліотеки; імена з `_` закриті (`iban._digits` — помилка). Закриті імена змінюйте вільно, відкриті — лише з новою версією.
- Перший рядок тіла функції — рядок-опис англійською, як коментарі коду: `"""IBAN in groups of four."""`. Його показує вихідний текст, надалі — підказки Studio (напрям ДЗ).

- Версія — константа `VERSION = "0.1.0"` у головному модулі й запис у README; несумісна зміна — нова старша цифра.
- Файл проєкту `project.p.prml` потрібен тестам: без нього корінь — каталог тесту, і `|import("../iban")` виходить за корінь (P2613). Проєкт, який підключає бібліотеку, цього файла не читає.
- Шляхи `load_*` і `|import` усередині бібліотеки — від файла модуля, без `../` за межі каталогу бібліотеки.

11.4. Модуль

```
---
kind: module
---
|code:
VERSION = "0.1.0"
_LETTERS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"


def normalized(text):
    """IBAN without spaces, in capitals."""
    return "".join(text.split()).upper()


def problem(text):
    """Why the text is not an IBAN UA, or "" if it is valid."""
    s = normalized(text)
    if not s.startswith("UA") or len(s) != 29 or not s[2:].isdigit():
        return "очікується UA і 27 цифр"
    digits = "".join([c if c.isdigit() else str(_LETTERS.find(c) + 10) for c in s[4:]
+ s[:4]])
    return "" if int(digits) % 97 == 1 else "не збігається контрольне число"


def check(text):
    """The normalized IBAN; a wrong one stops the build with the reason."""
    reason = problem(text)
    if reason:
        fail(f"IBAN «{text}»: {reason}")
    return normalized(text)
|end
```

Помилку даних документа бібліотека повідомляє через `fail()`: збирання зупиняється з повідомленням бібліотеки, місцем `fail` у модулі й рядком документа, звідки прийшов виклик. Для м'якої перевірки поруч дається функція, яка повертає причину (`problem`) або `True`/`False`.

11.5. Тести

Тест — звичайний документ: код порівнює результати з очікуваними й викликає `fail()` за першої розбіжності. Виклики на верхньому рівні `|code:` — дії, вони виконуються під час кожної перевірки.

```
|import("lib/iban")
|code:
def expect(name, got, wanted):
    if got != wanted:
        fail(f"{name}: отримано {repr(got)}, очікувалося {repr(wanted)}")


expect("check", iban.check("ua21 3223 1300 0002 6007 2335 6600 1"),
"UA213223130000026007233566001")
```

```
expect("problem", iban.problem("UA223223130000026007233566001"), "не збігається
контрольне число")
|end
Бібліотека ua-iban |(iban.VERSION).
```

`primula check tests/iban-test.prml` завершується кодом 0, якщо всі очікування збіглися, і кодом 2 з повідомленням `fail` інакше; `run-tests.sh` зразка перевіряє всі `tests/*.prml`. Перевіряйте граничні випадки й помилки, а не лише вдалий шлях: неправильна контрольна цифра, хибна довжина, чужа країна.

11.6. Підключення до проєкту

Три способи, від зручного для розробки до найбільш відтворюваного:

Спосіб	Як	Коли
символічне посилання	`ln -s ../shared/ua-iban lib` у корені проєкту	розробка, кілька проєктів на одній машині; збирання видає попередження P2639
копія	каталог бібліотеки копіюється в `lib/` проєкту	випуск документа: проєкт збирається однаково без зовнішніх каталогів
підмодуль Git	`git submodule add <url> lib/ua-iban`	проєкт у Git: версію бібліотеки закріплено комітом

```
cd мій-проєкт
ln -s ../shared/ua-iban lib

|import("lib/iban")
IBAN: |(iban.grouped(рахунок)) (код банку |(iban.bank_code(рахунок))).
```

Посилання перевіряється за записаним шляхом (рішення 136): `lib` усередині проєкту може вести будь-куди, а `../` у тексті документа й модуля, як і раніше, не виходить за корінь. Файли бібліотеки потрапляють до звіту збирання й до `--embed-sources` як `lib/iban.m.prml`, із гешами — за звітом видно, якою версією бібліотеки зібрано документ.

11.7. Дані з датою дії

Ставки, довідники й календарі змінюються з часом, а старий документ має збиратися так само, як під час випуску. Зберігайте значення з датою початку дії й вибирайте за датою, яку передає документ (дата документа, а не дата збирання):

```
vat:
  - {from: 2011-01-01, value: 20}

|code:
_rates = load_yaml("lib/data/rates.yaml")


def vat_on(day):
    """VAT rate on a date; the document passes doc.created or the contract date."""
    found = None
    for r in _rates.vat:
        if r["from"] <= day:
            found = r["value"]
    if found is None:
        fail(f"немає ставки на {fmt_date(day, 'short')}")
    return found
|end
Ставка ПДВ: |(vat_on(date(2026, 10, 26))) %.
```

Нове значення додається рядком із новою датою; старі рядки не змінюються.

11.8. Що всередині файла: primula describe

`primula describe FILE` показує зведення про файл, що не збирається в документ.

```
primula describe lib/ua_ids/ua_ids.m.prml        # функції, константи, версія модуля
primula describe styles/проєкт.s.prml            # extends, defaults, визначені стилі
primula describe project.p.prml --format json    # для редакторів і Studio
```

Для модуля виводяться початковий коментар `|%`, `VERSION`, відкриті функції із сигнатурою та першим рядком опису (docstring або коментар над `def`), відкриті константи й кількість закритих імен; код не виконується. Тому пишіть функціям docstring: він потрапляє до зведення. `primula view` показує те саме зведення для модуля, модуля стилів і проєкту.

11.9. Випуск

- README: призначення, функції з прикладами, версія, ліцензія, джерела правил (стандарт, закон, постанова) — і дата, на яку їх звірено.
- Перед випуском: `run-tests.sh` без помилок; номер у `VERSION` і в README збігається; зміни описано.
- Не додавайте до бібліотеки реальних персональних даних і реквізитів у прикладах — використовуйте свідомо навчальні значення.

Для розробки бібліотеки за допомогою AI є skill `skills/primula-library`: він веде ту саму схему — модуль, тести, перевірка транслятором.

11.10. Команди модуля: `|def`, `param`, накопичувач

Модуль може задавати не лише функції, а й **команди**: макрос `|def` викликається як `|модуль.команда(…)` (`|estimate.table(…)`, `|bib.cite(…)`). Ім'я макроса може збігатися з іменем вбудованої команди (`table`, `list`): виклик завжди з простором імен, вбудована команда не зачіпається.

- Опис команди та її параметрів — коментарі `|%` просто над `|def`; рядок `|% ім'я: текст` описує параметр. Studio показує їх у «Палітрі», підказці й доповненні, а `primula describe модуль` і `primula commands --document ФАЙЛ` віддають стороннім програмам.
- Налаштування бібліотеки беруться з параметрів документа функцією `param("ім'я", за_замовчуванням)` (наприклад, `currency` в `estimate`, `bib_order` у `bib`): макрос не бачить змінних документа, а параметри — бачить.
- Значення, що залежать від усього документа (номери посилань за порядком першого посилання, перелік термінів), збирає накопичувач: `collect("ім'я", значення)` у тексті й `collected("ім'я")` після всього документа; якщо `collected` викликається, документ обчислюється двічі (розділ 4.6 специфікації).

Зразки — приклади `examples/13-estimate` (кошторис), `14-bibliography` (посилання за порядком згадки) і `15-own-commands` (своя команда й словник термінів).

12. Імпорт документів із Word та інших форматів

Готовий договір, інструкція чи акт лежать у Word, ODT, Markdown або HTML. Команда `primula import` переводить їх у чернетку Primula: текст, заголовки, нумеровані пункти, списки, таблиці, зображення, виноски та посилання. Це початок роботи, а не її кінець: чернетку треба переглянути й довести — але передруковувати не потрібно.

Зразок — `examples/16-import`: три джерела (`договір.md`, `інструкція.html`, `акт.docx`) і результати імпорту поруч.

12.1. Команда й вікно Studio

```
primula import договір.docx                 # договір.prml і assets/ поруч
primula import інструкція.odt -o doc/інструкція.prml --language uk-UA
primula import договір.docx --list-styles   # лише перелічити стилі Word, нічого не
писати
```

У Studio — «Файл → Імпортувати документ…» (і «Імпортувати в Primula…» в контекстному меню файла): вікно питає шлях результату, мову, чи розпізнавати структуру, і показує зведення.

Джерело читає pandoc, тому він має бути встановлений (https://pandoc.org); старий двійковий `.doc` pandoc не читає — його спершу перетворює на DOCX LibreOffice. У збірці для Windows pandoc немає: там імпорт недоступний. Формат визначається за розширенням: `.docx`, `.odt`, `.doc`, `.rtf`, `.md`, `.html`, `.epub`, `.txt`; інший читач — `--from`.

Наявний результат не замінюється без `--force`. Після запису команда сама запускає `primula check` і друкує зведення: що перетворено, що пропущено та результат перевірки. Код повернення 2 — у чернетці лишилися помилки (файл усе одно записано).

12.2. Що переноситься

У джерелі	У Primula
абзаци, жирний, курсив, підкреслений, верхній і нижній індекс, закреслений	абзаци, `|*{}`, `|/{}`, `|_{}`, `|^{}`, `|,{}`, `|–{}`
заголовки	`|<`, `|<<`…
списки з вкладенням	`|-`, `|#`, `|--`, `|#-`… (абзаци всередині пункту — оточення `|list:`)
таблиці, об'єднання комірок, вирівнювання, ширини	`|table` з `|&(span=…, rows=…, align=…)`
зображення	`|img` з розмірами в мм, файли — в `assets/` (EMF і WMF Word — у SVG через LibreOffice)
виноски, посилання, цитати, код	`|footnote{}`, `|link`, `|.Quote:`, `|verbatim`
метадані	`doc.title`, `doc.creator`; назва береться ще з першого рядка ВЕЛИКИМИ літерами або з першого заголовка

Не переносяться й називаються у зведенні: верстка сторінки (колонтитули, поля, колонки), пряме форматування без стилю, виправлення й коментарі, поля Word, формули (лишається їхній текст), таблиця всередині комірки таблиці (текст склеюється), сира розмітка.

12.3. Як розпізнається структура

У документах із Word номери пунктів часто набрані руками, а заголовки — просто жирні рядки. Імпорт відновлює структуру, щоб номери й посилання потім робила Primula:

- **Набрані номери.** Абзац «2.1. Покупець оплачує…» стає пунктом `|>`, «2.1.1.» — `|>>`, короткий рядок «2. Ціна і порядок розрахунків» — заголовком `|<`. Номер прибирається з тексту: нумерацію дає Primula. Набрані номери звіряються з нумерацією результату; якщо вони не збіглися (пропущено «1.2», номер «4.» після «2.»), зведення каже скільки — номери результату йдуть за структурою.
- **Автонумерація Word.** Нумерований список з одного пункту — так Word і ODT зберігають нумерацію заголовків — стає заголовком (короткий текст) або пунктом (довге речення); вкладені списки — наступними рівнями.
- **Жирні рядки.** Короткий рядок цілком жирним після першого заголовка — заголовок наступного рівня; рядки титульної сторінки до першого заголовка («©…», назва) лишаються текстом.
- **Зміст.** Набраний зміст («Назва ….. 5» поспіль або список посилань під «Зміст») замінюється на `|toc()`.
- **Рівні без пропусків.** Перший заголовок — першого рівня, далі не глибше за «попередній + 1»; пункти — так само.

Правила евристичні: вони помиляються там, де автор документа не дотримувався жодних правил. Тому зведення називає, що зроблено, а `--no-structure` вимикає розпізнавання, `--keep-numbers` лишає набрані номери в тексті (заголовки тоді без номерів Primula, `number=False`).

12.4. Стилі Word

DOCX зберігає імена стилів абзаців і знаків, і імпорт зіставляє їх зі стилями Primula:

- заголовки Word («Heading 2», «Заголовок 2») — заголовки; `Clause` і `SubClause` — пункти (так повертається DOCX, зроблений `primula export`); «Quote», «Цитата» — `|.Quote:`; звичайні стилі («Normal», «Основний текст», «Hyperlink») — звичайний абзац; імена стилів Primula (`Body`, `Requisites`, `Strong`…) лишаються як є;
- власні стилі описуються в `import-styles.yaml` — поруч із результатом або в каталозі вище (або `--styles ФАЙЛ`):

```
paragraphs:
  "Пункт": {clause: 1}          # абзаци стилю «Пункт» — пункти |>
  "Заголовок договору": {heading: 1}
  "Реквізити": {style: Requisites}   # |.Requisites: …
  "Службовий": ignore           # абзац пропускається
characters:
  "Важливе": {style: Strong}    # |.Strong{…}
```

Значення: `{heading: N}`, `{clause: N}`, `{style: Ім'я}`, `plain` (звичайний абзац), `ignore`. Незіставлений стиль лишається звичайним абзацом і називається у зведенні; `--list-styles` показує всі стилі джерела, `--write-styles import-styles.yaml` пише заготовку для них. У ODT та інших форматів pandoc імен стилів не дає — там працюють лише правила структури.

12.5. Результат

Типова чернетка — документ без параметрів і посилань:

```
---
doc:
  title: "ДОГОВІР ПОСТАВКИ № 15"
  language: uk-UA
```

```
---
ДОГОВІР ПОСТАВКИ № 15

|toc()

|< Предмет договору

|> Постачальник зобов'язується поставити, а Покупець — прийняти й оплатити товар.

|< Ціна і порядок розрахунків

|> Покупець оплачує товар протягом 10 банківських днів.

|>> Зміна ціни можлива лише за згодою сторін.
```

12.6. Що зробити після імпорту

1. Виконати `primula check` (команда вже зробила це) і виправити попередження; зібрати `primula build` і подивитися результат.
2. Замінити посилання «п. 2.1» у тексті на `|@("id")` з `id=` у пункта (див. сторінку «Договір»): набрані номери в посиланнях самі не оновлюються.
3. Винести дані (сторони, дати, суми) в параметри заголовка чи проєкту й підставляти `|(ім'я)`.
4. Призначити стилі з набору комплекту (сторінка «Стилі») замість `|.Body:` і звичайних абзаців; підписи та реквізити — таблицею або стилем.
5. Якщо документів багато — скласти `import-styles.yaml` за результатами `--list-styles` і імпортувати решту тим самим файлом.
6. Для вичитки й доопрацювання результату — AI у Studio («Запитати AI…» з рецептами «вичитка», «терміни договору»): правки застосовуються з перевіркою транслятором.

Межі: імпорт — чернетка. Він не вгадує змісту (який рядок — реквізити, а який — умова), не знає верстки й не замінює вичитки.

Додаток A. Конструкції

Згенеровано `tools/author_docs.py appendices` з довідника транслятора; не правити вручну.

Команди

Текст

Запис	Призначення	Аргументи	§
`|link`	зовнішнє посилання; без тіла текст — сама адреса	`url`	4.2
`|fmt`	пряме форматування фрагмента (попередження: краще символьні стилі)	`**props`	4.4
`|set`	стилі конструкцій до кінця охопного оточення	`**keys`	4.5
`|block`	стиль блока для групи елементів	`style`, `when`	4.3
`|list`	стиль одного списку: рядок перед першим елементом або оточення	`style`	5.4

Структура

Запис	Призначення	Аргументи	§
`|appendix`	додаток: позначення, назва, власна область нумерації, нова сторінка	`id`, `title`, `subtitle`, `number`, `letter`, `page`, `block`, `toc`, `toc_title`, `toc_level`, `when`	5.5
`|scope`	область нумерації з незалежними лічильниками	`name`, `prefix`	5.5
`|toc`	зміст (покажчик, що оновлює Writer)	`max_level`, `title`, `page_break`, `track`	5.9
`|list_of`	перелік рисунків, таблиць, скорочень або абетковий покажчик	`kind`, `title`, `page_break`, `track`	5.9
`|toc_entry`	рядок змісту із закладкою на наступний абзац	`text`, `level`	5.9
`|footnote`	виноска внизу сторінки	`id`	5.10
`|endnote`	кінцева виноска	`id`	5.10

Таблиці

Запис	Призначення	Аргументи	§
`|table`	таблиця: рядки |: … |& … або рядки з даних	`cols`, `style`, `width`, `align`, `header`, `footer`, `repeat_header`, `keep_rows`, `may_break`, `frame`, `rules`, `caption`, `id`, `block`, `when`, `data`, `fields`, `titles`, `live`	6.2
`|row`	рядок таблиці в розгорнутій формі	`align`, `valign`, `padding`, `background`, `paragraph`, `border`, `height`, `min_height`, `keep`, `when`	6.4
`|cell`	комірка таблиці в розгорнутій формі	`span`, `rows`, `align`, `valign`, `padding`, `background`, `paragraph`, `border`	6.4
`|caption`	підпис таблиці з розміткою		5.8

Сторінки й графіка

Запис	Призначення	Аргументи	§
`|page`	зміна стилю сторінки: звідси або для оточення	`style`, `restart`, `number_format`	7.3
`|newpage`	розрив сторінки		7.4
`|newcolumn`	перехід до наступної колонки		7.4
`|section`	розділ з колонками	`cols`, `widths`, `gap`, `rule`, `balance`, `id`, `block`, `when`	7.5

Запис	Призначення	Аргументи	§
`|header`	верхній колонтитул стилю сторінки	`page`, `which`	7.6
`|footer`	нижній колонтитул стилю сторінки	`page`, `which`	7.6
`|field.page`	номер сторінки (поле Writer)		7.6
`|field.pages`	кількість сторінок (поле Writer)		7.6
`|field.chapter`	назва поточного заголовка (поле Writer)	`level`	7.6
`|field.title`	назва документа з метаданих (поле Writer)		7.6
`|img`	зображення без підпису	`path`, `key`, `svg`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.2
`|figure`	нумерований рисунок; тіло — підпис	`path`, `key`, `svg`, `id`, `block`, `caption`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.3
`|frame`	текстовий блок	`id`, `block`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.4
`|watermark`	водяний знак на всіх або перелічених стилях сторінок	`mark`, `pages`, `angle`, `color`, `size`, `opacity`, `font`	8.6
`|parallel`	паралельні потоки: тексти поруч, сегменти яких починаються на одному рівні (двомовний договір, стара й нова редакція, текст і коментар)	`langs`, `tracks`, `primary`, `widths`, `gap`, `rule`, `header`, `parity`, `style`, `keep_rows`, `id`	7.7
`|segment`	сегмент |parallel: блок на кожен потік, початки на одному рівні	`id`, `span`, `parity`, `keep`, `status`, `when`	7.7
`|track`	блок потоку в сегменті; закінчується наступним |track або кінцем сегмента	`track`, `status`, `rev`, `when`	7.7

Технічна документація

Запис	Призначення	Аргументи	§
`|abbr`	скорочення: визначення й вживання або вживання	`short`, `full`, `text`, `show`	9.4
`|index`	елемент абеткового покажчика	`term`, `sub`	9.5

Фрагменти, модулі, частини

Запис	Призначення	Аргументи	§
`|include`	вставити фрагмент	`path`, `part`, `key`, `ns`, `rename`, `level_shift`, `**params`	10.2
`|import`	підключити модуль визначень	`path`, `*names`, `alias`	10.3
`|document`	частина зведеного документа (своя область, параметри, нова сторінка)	`path`, `scope`, `params`, `page`, `level_shift`, `when`	10.7
`|part`	іменована частина фрагмента для |include(part=…)	`name`	10.2
`|export`	експортувати імена фрагмента в документ	`*names`, `**names`	10.4

Код і дані

Запис	Призначення	Аргументи	§
`|let`	визначити імена	`**names`, `export`, `override`	10.4
`|code`	блок коду обчислювального ядра		3.7

Запис	Призначення	Аргументи	§
`|comment`	блоковий коментар		3.7
`|verbatim`	буквальний блок	`end`	3.7
`|if`	умовний вміст	`condition`	9.2
`|elif`	інша гілка |if	`condition`	9.2
`|else`	остання гілка |if		9.2
`|for`	цикл за списком	`loop`	6.9
`|def`	макрос: команда користувача	`signature`	3.2
`|call`	динамічний виклик функції як команди	`function`, `*args`	3.2

Короткі форми

Запис	Призначення	§
`||`	літерал |	3.3
`|{ |}`	літеральні фігурні дужки	3.3
`|;`	порожній роздільник: межа імені команди	3.3
`|%`	коментар до кінця рядка	3.7
`|(…)`	значення виразу як текст	4.2
`|\`	розрив рядка в абзаці	3.3
`|~`	нерозривний пробіл	3.3
`|*{…}`	напівжирний	3.3
`|/{…}`	курсив	3.3
`|_{…}`	підкреслення	3.3
`|^{…}`	верхній індекс	3.3
`|,{…}`	нижній індекс	3.3
`|–{…}`	закреслення	3.3
|{…}`	моноширинний: позначення, коди	3.3
`|"{…}`	лапки мови документа	4.2
`|?{…}`	заповнювач чернетки	4.2
`|.Ім'я{…}`	символьний стиль	4.3
`|.Ім'я:`	абзацний стиль (рядок або оточення |end.Ім'я)	4.3
`|@("id")`	перехресне посилання: номер, label, title, page	5.7
`|<`	заголовок; кількість < — рівень (1–6) або level=	5.2
`|>`	пункт; кількість > — рівень (оточення |end>)	5.3
`|-`	маркований елемент списку; довжина — рівень	5.4
`|#`	нумерований елемент списку; довжина — рівень	5.4
`|!("kind")`	попередження або примітка: danger, warning, caution, notice, note, tip	9.1
`|:`	рядок таблиці	6.4
`|&`	комірка таблиці	6.4
`|=(…)`	відкладена формула комірки	6.8

Маркери закриття

Запис	Призначення	§
`|end`	закриває найближче оточення без перевірки	3.5
`|end name`	закриває іменоване оточення з перевіркою	3.5
`|end>`	закриває пункт цього рівня	3.5

Запис	Призначення	§
`|end!`	закриває попередження	3.5
`|end.Ім'я`	закриває абзацний стиль	3.5
`|endif`	закриває |if	3.5
`|endfor`	закриває |for	3.5
`|enddef`	закриває |def	3.5

Додаток Б. Функції виразів

Згенеровано `tools/author_docs.py appendices` з довідника транслятора; не правити вручну.

Числа

Виклик	Призначення	Приклад → результат	§
`num(x, digits=None, lang)`	число за правилами мови: групи розрядів, кома	`num(12500, digits=2) → "12 500,00"`	бібл. 2
`percent(x, digits=None, lang)`	відсоток з нерозривним пробілом: «20 %»	`percent(20) → "20 %"`	бібл. 2
`round_to(x, step)`	округлити до кроку, половина від нуля	`round_to(1234, 50) → 1250`	бібл. 2
`ceil_to(x, step)`	округлити до кроку вгору	`ceil_to(1201, 100) → 1300`	бібл. 2
`floor_to(x, step)`	округлити до кроку вниз	`floor_to(1299, 100) → 1200`	бібл. 2
`round_money(x)`	округлити до сотих, половина від нуля	`round_money(dec("10.005")) → 10.01`	бібл. 2
`round(x, ndigits=None)`	округлити, половина від нуля; без ndigits — ціле	`round(2.675, 2) → 2.68`	12.4
`abs(x)`	модуль числа	`abs(-5) → 5`	12.10
`min(*values, key, default)`	найменше з аргументів або списку	`min(3, 1, 2) → 1`	12.10
`max(*values, key, default)`	найбільше з аргументів або списку	`max([3, 1, 2]) → 3`	12.10
`sum(seq, start=0)`	сума списку	`sum([1.50, 2.25]) → 3.75`	12.10
`dec(x)`	у decimal (з float — найкоротший десятковий запис)	`dec(0.1) → 0.1`	12.4
`float(x)`	у float; datetime дає серійне число	`float("2.5") → 2.5`	12.4
`int(x)`	відкинути дробову частину (decimal з масштабом 0)	`int(7.9) → 7`	12.4

Гроші й ПДВ

Виклик	Призначення	Приклад → результат	§
`money(x, code, currency_style="short", lang)`	сума з валютою: «12 500,00 грн»	`money(12500, "UAH") → "12 500,00 грн"`	бібл. 3
`money_words(x, code, kop="digits", cap=False, lang)`	сума прописом	`money_words(12500, "UAH") → "дванадцять тисяч п'ятсот гривень 00 копійок"`	бібл. 3
`money_full(x, code, kop="digits", cap=False, currency_style="short", lang)`	сума цифрами і прописом у дужках (договірна форма)	`money_full(12500.50, "UAH") → "12 500,50 грн (дванадцять тисяч п'ятсот гривень 50 копійок)"`	бібл. 3
`vat(x, rate)`	податок із суми без податку	`vat(1000, 20) → 200.00`	бібл. 3
`with_vat(x, rate)`	сума з податком	`with_vat(1000, 20) → 1200.00`	бібл. 3
`vat_in(x, rate)`	податок, що міститься в сумі з податком	`vat_in(1200, 20) → 200.00`	бібл. 3

Виклик	Призначення	Приклад → результат	§
`without_vat(x, rate)`	сума без податку із суми з податком	`without_vat(1200, 20) → 1000.00`	бібл. 3
`split(total, shares)`	поділити суму за частками; частини дають суму	`split(100, [1, 1, 1]) → [33.33, 33.33, 33.34]`	бібл. 3

Прописом

Виклик	Призначення	Приклад → результат	§
`words(x, code=None, gender="m", case="nom", lang)`	число прописом; words(x, "UAH") — сума	`words(21, gender="f") → "двадцять одна"`	бібл. 4
`ordinal(n, gender="m", case="nom", lang)`	порядковий числівник прописом	`ordinal(21, case="gen") → "двадцять першого"`	бібл. 4
`date_words(dt, lang)`	дата прописом	`date_words(date(2026, 10, 12)) → "дванадцятого жовтня дві тисячі двадцять шостого року"`	бібл. 4

Дати й час

Виклик	Призначення	Приклад → результат	§
`date(y, m, d)`	дата: date(y, m, d) або date("YYYY-MM-DD")	`date(2026, 10, 12) → date("2026-10-12")`	12.3
`datetime(y, m, d, h=0, mi=0, s=0)`	момент: datetime(y, m, d, h, mi, s), рядок ISO або серійне число	`datetime("2026-10-12T14:30") → datetime("2026-10-12T14:30:00")`	12.3
`build_date()`	дата початку збирання (--build-date, build_date: проєкту)	`build_date() → date("2026-10-12")`	12.7
`weeks(n)`	кількість діб у n тижнях: dt + weeks(2)	`weeks(2) → 14`	бібл. 5
`hours(n)`	кількість діб у n годинах: dt + hours(36)	`hours(36) → 1.5`	бібл. 5
`minutes(n)`	кількість діб у n хвилинах	`date(2026, 10, 12) + minutes(90) → datetime("2026-10-12T01:30:00")`	бібл. 5
`add_months(dt, n)`	зсув на місяці (31 → останній день місяця)	`add_months(date(2026, 1, 31), 1) → date("2026-02-28")`	бібл. 5
`add_years(dt, n)`	зсув на роки	`add_years(date(2028, 2, 29), 1) → date("2029-02-28")`	бібл. 5
`fmt_date(dt, format="short", lang)`	дата текстом: short, long, contract або шаблон (ДСТУ 4163:2020)	`fmt_date(date(2026, 10, 12), "contract") → "«12» жовтня 2026 р."`	бібл. 5
`fmt_time(dt, format="short", lang)`	час текстом: short або шаблон {hh}:{mi}	`fmt_time(datetime(2026, 10, 12, 14, 30, 0)) → "14:30"`	бібл. 5
`fmt_datetime(dt, format="short", lang)`	дата й час текстом	`fmt_datetime(datetime(2026, 10, 12, 14, 30, 0), "long") → "12 жовтня 2026 року, 14:30"`	бібл. 5
`weekday(dt, case="nom", lang)`	назва дня тижня (відмінок nom або acc)	`weekday(date(2026, 10, 12)) → "понеділок"`	бібл. 5
`month_name(dt, case="nom", lang)`	назва місяця (відмінок nom, gen, loc)	`month_name(date(2026, 10, 12), case="gen") → "жовтня"`	бібл. 5
`is_workday(dt, calendar)`	чи робочий день (за календарем проєкту)	`is_workday(date(2026, 10, 10)) → False`	бібл. 5
`workdays(dt, n, calendar)`	дата через n робочих днів після dt (n < 0 — до)	`workdays(date(2026, 10, 9), 1) → date("2026-10-12")`	бібл. 5

Одиниці й довжини

Виклик	Призначення	Приклад → результат	§
`qty(x, unit, tol, digits=None, lang)`	значення з одиницею: «230 В», tol= — допуск	`qty(230, "V") → "230 В"`	бібл. 6
`qty_range(low, high, unit, digits=None, lang)`	діапазон з одиницею: «100–240 В»	`qty_range(-20, 55, "degC") → "−20…+55 °C"`	бібл. 6
`convert(x, from, to)`	перерахувати між одиницями однієї величини	`convert(1.5, "kW", "W") → 1500`	бібл. 6
`length(text)`	довжина з рядка: length("10mm")	`length("10mm") + 5*mm → length("15mm")`	12.3
`mm`	константа довжини: 10*mm	`10*mm → length("10mm")`	12.3
`cm`	константа довжини: 10*cm	`10*cm → length("10cm")`	12.3
`pt`	константа довжини: 10*pt	`10*pt → length("10pt")`	12.3
`inch`	константа довжини: 10*inch	`10*inch → length("10inch")`	12.3

Текст

Виклик	Призначення	Приклад → результат	§
`plural(n, *forms, frac, lang)`	форма слова за числом: plural(n, "день", "дні", "днів")	`plural(5, "день", "дні", "днів", lang="uk") → "днів"`	бібл. 7
`join_list(items, last, sep=", ", lang)`	з'єднати список: «А, Б та В»	`join_list(["А", "Б", "В"], last="та") → "А, Б та В"`	бібл. 7
`cap(s)`	велика перша літера	`cap("київ") → "Київ"`	бібл. 7
`translit(s, lang)`	транслітерація (uk — постанова КМУ № 55, ru — ICAO Doc 9303)	`translit("Згорани", lang="uk") → "Zghorany"`	бібл. 7
`sort_key(s, lang)`	ключ сортування за абеткою мови: sorted(names, key=sort_key)	`sorted(["Ґава", "Гора", "Єва", "Есе"], key=sort_key) → ['Гора', 'Ґава', 'Есе', 'Єва']`	бібл. 7
`nbsp_short(s, lang)`	нерозривний пробіл після коротких прийменників і сполучників	`nbsp_short("в домі та у дворі", lang="uk") → "в домі та у дворі"`	бібл. 7
`str(x)`	значення рядком (дати — ISO, числа — з крапкою)	`str(2.20) → "2.20"`	12.10
`repr(x)`	значення у вигляді коду	`repr("a") → "'a'"`	12.10

Списки й записи

Виклик	Призначення	Приклад → результат	§
`len(seq)`	довжина: символи, елементи, ключі	`len("Київ") → 4`	12.10
`range(start, stop, step=1)`	числа від start до stop (не включно) з кроком	`range(1, 10, 3) → [1, 4, 7]`	12.10
`enumerate(seq, start=0)`	пари (номер, елемент)	`enumerate(["a", "b"], start=1) → [(1, 'a'), (2, 'b')]`	12.10
`zip(*lists)`	кортежі елементів з однаковим індексом	`zip([1, 2], ["a", "b"]) → [(1, 'a'), (2, 'b')]`	12.10
`sorted(seq, key, reverse=False)`	відсортований список	`sorted([3, 1, 2]) → [1, 2, 3]`	12.10
`reversed(seq)`	список у зворотному порядку	`reversed([1, 2, 3]) → [3, 2, 1]`	12.10
`any(seq)`	чи є хоч один істинний елемент	`any([0, 1]) → True`	12.10

Виклик	Призначення	Приклад → результат	§
`all(seq)`	чи всі елементи істинні	`all([1, 2]) → True`	12.10
`list(seq)`	список (з послідовності)	`list("abc") → ['a', 'b', 'c']`	12.10
`tuple(seq)`	кортеж	`tuple([1, 2]) → (1, 2)`	12.10
`dict(source=None, **items)`	словник: з пар, запису або key=value	`dict(a=1, b=2) → {'a': 1, 'b': 2}`	12.10
`record(source=None, **fields)`	запис з доступом через крапку: r.name	`record(name="Кабель", qty=3) → record(name='Кабель', qty=3)`	12.6
`bool(x)`	логічне значення	`bool("") → False`	12.10
`getattr(r, name, default)`	поле запису за ім'ям, зі значенням за замовчуванням	`getattr(record(a=1), "b", 0) → 0`	12.6
`hasattr(r, name)`	чи має запис поле	`hasattr(record(a=1), "a") → True`	12.6
`type(x)`	ім'я типу: "decimal", "str", "datetime"…	`type(1.5e3) → "float"`	12.3

Дані

Виклик	Призначення	Приклад → результат	§
`group_by(items, key)`	словник «значення ключа → список записів»	`group_by([record(k="a", v=1), record(k="a", v=2)], "k") → {'a': [record(k='a', v=1), record(k='a', v=2)]}`	бібл. 8
`sum_by(items, key, value)`	суми поля за групами	`sum_by([record(k="a", v=1), record(k="a", v=2)], "k", "v") → {'a': 3}`	бібл. 8
`distinct(items, key)`	унікальні значення поля	`distinct([record(k="a"), record(k="b"), record(k="a")], "k") → ['a', 'b']`	бібл. 8
`load_csv(path, types, sep)`	рядки файлу CSV як записи	`load_csv("data/items.csv")`	10.5
`load_yaml(path)`	дані файлу YAML (записи й списки)	`load_yaml("data/order.yaml")`	10.5
`load_json(path)`	дані файлу JSON	`load_json("data/order.json")`	10.5
`load_text(path)`	увесь текстовий файл як рядок (UTF-8; наприклад, SVG для |img(svg=…))	`load_text("img/sign.svg")`	10.5
`load_ods(path, sheet, range, header=True, types)`	рядки аркуша файлу ODS як записи (збережені результати формул)	`load_ods("data/items.ods", sheet="Позиції")`	10.5
`load_xlsx(path, sheet, range, header=True, types)`	рядки аркуша файлу XLSX як записи (збережені результати формул)	`load_xlsx("data/items.xlsx", sheet="Позиції")`	10.5

Математика (std/math)

Виклик	Призначення	Приклад → результат	§
`math.sqrt(x)`	квадратний корінь; у документі: |import("std/math")	`math.sqrt(2) → 1.4142135623730951`	бібл. 9
`math.exp(x)`	e в степені x; у документі: |import("std/math")	`math.exp(1) → 2.718281828459045`	бібл. 9
`math.log(x)`	натуральний логарифм; у документі: |import("std/math")	`math.log(math.e) → 1.0`	бібл. 9
`math.log10(x)`	десятковий логарифм; у документі: |import("std/math")	`math.log10(1000) → 3.0`	бібл. 9
`math.sin(x)`	синус (радіани); у документі: |import("std/math")	`math.sin(math.pi / 2) → 1.0`	бібл. 9
`math.cos(x)`	косинус (радіани); у документі: |import("std/math")	`math.cos(0) → 1.0`	бібл. 9

Виклик	Призначення	Приклад → результат	§
`math.tan(x)`	тангенс (радіани); у документі: |import("std/math")	`math.tan(0) → 0.0`	бібл. 9
`math.atan2(y, x)`	кут точки (x, y), радіани; у документі: |import("std/math")	`math.atan2(1, 1) → 0.7853981633974483`	бібл. 9
`math.floor(x)`	округлити вниз до цілого (decimal); у документі: |import("std/math")	`math.floor(-2.5) → -3`	бібл. 9
`math.ceil(x)`	округлити вгору до цілого (decimal); у документі: |import("std/math")	`math.ceil(2.1) → 3`	бібл. 9
`math.pi`	число π (float); у документі: |import("std/math")	`math.pi → 3.141592653589793`	бібл. 9
`math.e`	число e (float); у документі: |import("std/math")	`math.e → 2.718281828459045`	бібл. 9

QR-коди й кодування

Виклик	Призначення	Приклад → результат	§
`qr_matrix(data, ecc="M", min_version=1, max_version=40, encoding="utf-8")`	QR-код тексту (байтовий режим): {version, ecc, mask, size, rows}	`qr_matrix("https://primula.example").version → 2`	бібл. 10
`qr_svg(data, ecc="M", min_version=1, max_version=40, encoding="utf-8", size="30mm", quiet=4, circle=0)`	QR-код як SVG для |img(svg=…); circle= лишає біле коло в центрі	`len(qr_svg("https://primula.example")) > 0 → True`	бібл. 10
`base64url(text, encoding="utf-8")`	Base64URL байтів тексту, без доповнення =	`base64url("Київ") → "0JrQuNGX0LI"`	бібл. 10

Перевірки й документ

Виклик	Призначення	Приклад → результат	§
`fail(*message)`	зупинити збирання з помилкою в цьому місці	`fail("no price")`	12.9
`debug(*values)`	записати значення у звіт збирання, а не в документ	`debug(total)`	12.9
`has(feature)`	чи має варіант виробу функцію (keys: проєкту, --feature)	`has("wifi")`	9.2
`markup(text)`	розмітка Primula з рядка (content)	`markup("|*{" + name + "}")`	4.2
`param(name, default)`	параметр документа за назвою (і в модулі: його налаштування), або значення за замовчуванням (рішення 144)	`param("currency", "UAH")`	10.6
`collect(name, value)`	додати значення до накопичувача документа; результат — порожній вміст (рішення 143)	`|(collect("cited", key))`	4.6
`collected(name)`	значення накопичувача після всього документа, у порядку collect(); документ обчислюється двічі, у першому проході [] (рішення 143)	`collected("cited")`	4.6
`img(path, svg, **options)`	значення-зображення для watermark=, |img(значення), |figure(значення); шлях або svg=	`img("assets/stamp.png")`	8.6
`col(width, id, type, **props)`	стовпець таблиці: col(width, id=…, type=…, align=…)	`col("20mm..35mm", id="sum", type="money", align="right")`	6.3
`track(id, lang, title, **props)`	потік |parallel: track(id, lang=…, title=…, width=…, primary=…)	`track("note", title="Коментар", width="28%", numbered=False, optional=True)`	7.7

Формули таблиць

Виклик	Призначення	Приклад → результат	§
`this_row(c)`	значення в стовпці с поточного рядка	`this_row("qty") * this_row("price")`	6.8
`cell(r, c)`	значення клітинки	`cell(2, "sum")`	6.8
`column(c, rows)`	список значень стовпця (рядки тіла або rows=)	`sum(column("sum"))`	6.8
`body_rows()`	номери рядків між заголовком і підсумками	`body_rows()`	6.8
`avg(seq)`	середнє значення списку	`avg(column("qty"))`	6.8
`count(seq)`	кількість непорожніх значень	`count(column("qty"))`	6.8

Код стилів

Виклик	Призначення	Приклад → результат	§
`paragraph_style(name, parent, **props)`	визначити абзацний стиль: ім'я, parent=…, властивості (11.3)	`paragraph_style("MyStyle", parent="…")`	11.2
`character_style(name, parent, **props)`	визначити символьний стиль: ім'я, parent=…, властивості (11.3)	`character_style("MyStyle", parent="…")`	11.2
`block_style(name, parent, **props)`	визначити блоковий стиль: ім'я, parent=…, властивості (11.3)	`block_style("MyStyle", parent="…")`	11.2
`table_style(name, parent, **props)`	визначити табличний стиль: ім'я, parent=…, властивості (11.3)	`table_style("MyStyle", parent="…")`	11.2
`list_style(name, parent, **props)`	визначити списковий стиль: ім'я, parent=…, властивості (11.3)	`list_style("MyStyle", parent="…")`	11.2
`frame_style(name, parent, **props)`	визначити фреймовий стиль: ім'я, parent=…, властивості (11.3)	`frame_style("MyStyle", parent="…")`	11.2
`page_style(name, parent, **props)`	визначити сторінковий стиль: ім'я, parent=…, властивості (11.3)	`page_style("MyStyle", parent="…")`	11.2
`derive(base, name, **props)`	похідний стиль: його детерміноване ім'я для |.(…)	`derive("Clause", left="10mm")`	11.4
`structure_styles()`	стилі конструкцій структури: paragraph=, heading=, clause=…	`structure_styles(paragraph="Body")`	11.5
`structure_profile()`	профіль структури: "business" або "report"	`structure_profile("business")`	5.1
`numbering(headings, heading_format, clause_format, figures, tables, ref_format)`	нумерація заголовків, пунктів, рисунків і таблиць	`numbering(figures="by_chapter")`	5.2
`captions(lang, figure, table)`	слова підписів рисунків і таблиць	`captions("uk", figure="Рисунок", table="Таблиця")`	5.8
`appendices(lang, designation, numbering, letters, page_break, toc, title_format)`	позначення й оформлення додатків	`appendices("uk", designation="ДОДАТОК")`	5.9
`ref_labels(lang)`	слова посилань |@(id, "label")	`ref_labels("uk", clause="п.")`	5.7
`footnotes(numbering, restart, format, separator)`	нумерація й роздільник виносок	`footnotes(restart="page")`	5.10
`abbreviations(first, sort)`	перше вживання й сортування скорочень	`abbreviations(first="expand")`	9.4
`typography(percent_space, nbsp_short, nbsp_words)`	типографіка: percent_space, nbsp_short, nbsp_words	`typography(nbsp_short=True)`	бібл. 7

Виклик	Призначення	Приклад → результат	§
`admonition_symbols(danger, warning, caution, notice, note, tip)`	зображення знаків безпеки попереджень	`admonition_symbols(danger="assets/signs/w012.svg")`	9.1
`admonition_words(lang, danger, warning, caution, notice, note, tip, notes)`	сигнальні слова попереджень	`admonition_words("uk", danger="НЕБЕЗПЕКА")`	9.1
`review_marks(lang, true, false, watermark)`	тексти позначок і водяного знака -- review	`review_marks("uk", watermark="НЕ ДЛЯ ВИПУСКУ")`	9.2
`watermark()`	водяний знак на стилях сторінок	`watermark("ПРОЄКТ")`	8.6
`header()`	верхній колонтитул стилю сторінки	`header("Portrait", markup("…"))`	7.6
`footer()`	нижній колонтитул стилю сторінки	`footer("Portrait", markup("…"))`	7.6
`placeholders()`	заповнювачі |?{…}: "mark" (чернетка) або "final"	`placeholders("final")`	4.2
`quotes()`	лапки мови для |"{…}	`quotes("de", ("„", "“"), ("‚", "‘"))`	4.2
`currency(code, lang, short, units, frac, gender, minor, minor_gender, minor_digits)`	додати або змінити валюту бібліотеки	`currency("PLN", "uk", short="zł")`	бібл. 1
`unit(unit, lang, symbol, quantity, factor, offset)`	додати або змінити одиницю бібліотеки	`unit("rpm", "uk", symbol="об/хв", quantity="frequency", factor=1/60)`	бібл. 1
`month_names(lang, nom, gen, loc)`	назви місяців мови	`month_names("uk", gen=["січня", …])`	бібл. 1

Проєкт

Виклик	Призначення	Приклад → результат	§
`build(source, output, params, keys, pdf)`	зареєструвати ціль збирання проєкту	`build("act.prml", params=row, output=f"out/act-{row.no}.odt")`	10.7
`batch_mode()`	True під час збирання всього проєкту	`batch_mode()`	10.6

Методи

Виклик	Призначення	Приклад → результат	§
`str.split(sep=None, maxsplit=-1)`	розбити за роздільником (без нього — за пробілами)	`"a, b".split(", ") → ['a', 'b']`	12.10
`str.rsplit(sep=None, maxsplit=-1)`	розбити справа	`"a.b.c".rsplit(".", 1) → ['a.b', 'c']`	12.10
`str.splitlines(keepends=False)`	список рядків	`"a\nb".splitlines() → ['a', 'b']`	12.10
`str.join(items)`	з'єднати рядки цим рядком	`", ".join(["a", "b"]) → "a, b"`	12.10
`str.strip(chars=None)`	без пробілів (або символів) з обох кінців	`"  a ".strip() → "a"`	12.10
`str.lstrip(chars=None)`	без пробілів на початку	`"  a".lstrip() → "a"`	12.10
`str.rstrip(chars=None)`	без пробілів у кінці	`"a  ".rstrip() → "a"`	12.10
`str.replace(old, new, count=-1)`	замінити підрядок	`"a-b".replace("-", "/") → "a/b"`	12.10
`str.startswith(prefix)`	чи починається з префікса	`"ДСТУ 3008".startswith("ДСТУ") → True`	12.10

Виклик	Призначення	Приклад → результат	§
`str.endswith(suffix)`	чи закінчується суфіксом	`"act.prml".endswith(".prml") → True`	12.10
`str.find(sub, start=None, end=None)`	індекс підрядка або −1	`"abc".find("c") → 2`	12.10
`str.rfind(sub, start=None, end=None)`	індекс останнього входження або −1	`"abca".rfind("a") → 3`	12.10
`str.count(sub, start=None, end=None)`	кількість входжень	`"banana".count("a") → 3`	12.10
`str.upper()`	великими літерами	`"київ".upper() → "КИЇВ"`	12.10
`str.lower()`	малими літерами	`"КИЇВ".lower() → "київ"`	12.10
`str.capitalize()`	перша велика, решта малі	`"кИЇВ".capitalize() → "Київ"`	12.10
`str.title()`	кожне слово з великої	`"нова пошта".title() → "Нова Пошта"`	12.10
`str.isdigit()`	чи лише цифри	`"2026".isdigit() → True`	12.10
`str.isalpha()`	чи лише літери	`"Київ".isalpha() → True`	12.10
`str.isspace()`	чи лише пробіли	`" ".isspace() → True`	12.10
`str.zfill(width)`	доповнити нулями зліва	`"7".zfill(3) → "007"`	12.10
`str.format(*args, **kwargs)`	підставити значення: "{0} — {name}"	`"{0} шт.".format(3) → "3 шт."`	12.10
`list.append(x)`	додати елемент (усередині функцій)	`[1].append(2)`	12.10
`list.extend(seq)`	додати елементи списку	`[1].extend([2, 3])`	12.10
`list.insert(i, x)`	вставити елемент за індексом	`[1].insert(0, 0)`	12.10
`list.pop(i=-1)`	вилучити й повернути елемент (останній)	`[1, 2].pop() → 2`	12.10
`list.remove(x)`	вилучити перший рівний елемент	`[1, 2].remove(1)`	12.10
`list.index(x)`	індекс елемента	`["a", "b"].index("b") → 1`	12.10
`list.count(x)`	кількість рівних елементів	`[1, 1, 2].count(1) → 2`	12.10
`list.sort(key, reverse=False)`	відсортувати на місці	`[2, 1].sort()`	12.10
`list.reverse()`	обернути на місці	`[1, 2].reverse()`	12.10
`dict.get(key, default=None)`	значення за ключем або типове	`{"a": 1}.get("b", 0) → 0`	12.10
`dict.keys()`	список ключів	`{"a": 1}.keys() → ['a']`	12.10
`dict.values()`	список значень	`{"a": 1}.values() → [1]`	12.10
`dict.items()`	пари (ключ, значення)	`{"a": 1}.items() → [('a', 1)]`	12.10
`dict.update(other)`	додати пари іншого словника	`{"a": 1}.update({"b": 2})`	12.10
`dict.pop(key, default)`	вилучити ключ і повернути значення	`{"a": 1}.pop("a") → 1`	12.10
`dict.setdefault(key, default=None)`	значення за ключем; відсутнє — встановити типове	`{}.setdefault("a", 1) → 1`	12.10

Додаток B. Ключі YAML-заголовка

Згенеровано `tools/author_docs.py appendices` з довідника транслятора; не правити вручну.

Ключ	Файли	Тип	Призначення	Приклад → результат	§
`kind`	document, fragment, module, styles, project	string	вид файлу; випливає з підрозширення (.f, .m, .s, .p), у проєкті обов'язковий	`kind: document`	2.2
`primula`	document, fragment, module, styles, project	mapping	оголошення мови: {version: "1.1"}, за потреби версія бібліотеки	`primula: {version: "1.1"}`	2.3
`primula.version`	document, fragment, module, styles, project	string	версія мови; приймається лише 1.1	`version: "1.1"`	2.3
`primula.stdlib`	document, fragment, module, styles, project	string	мінімальна версія стандартної бібліотеки	`stdlib: "1.1"`	13
`doc`	document	mapping	метадані ODT документа; у тексті |(doc.title)	`doc:`	2.3
`doc_defaults`	project	mapping	типові значення метаданих (doc:) усіх документів проєкту; документ перевизначає їх поштучно	`doc_defaults:`	10.6
`doc.title`	document	string	назва (dc:title) (рекомендований)	`title: "Договір поставки"`	2.3
`doc_defaults.title`	project	string	назва (dc:title)	`title: "Договір поставки"`	10.6
`doc.subject`	document	string	тема (dc:subject)	`subject: "Поставка обладнання"`	2.3
`doc_defaults.subject`	project	string	тема (dc:subject)	`subject: "Поставка обладнання"`	10.6
`doc.description`	document	string	опис (dc:description)	`description: "Типова форма"`	2.3
`doc_defaults.description`	project	string	опис (dc:description)	`description: "Типова форма"`	10.6
`doc.creator`	document	string	автор (meta:initial-creator, dc:creator) (рекомендований)	`creator: "Іван Петренко"`	2.3
`doc_defaults.creator`	project	string	автор (meta:initial-creator, dc:creator) (рекомендований)	`creator: "Іван Петренко"`	10.6
`doc.organization`	document	string	організація (користувацька властивість)	`organization: "ТОВ «Постачальник»"`	2.3

Ключ	Файли	Тип	Призначення	Приклад → результат	§
`doc_defaults.organization`	project	string	організація (користувацька властивість) (рекомендований)	`organization: "ТОВ «Постачальник»"`	10.6
`doc.language`	document	language	мова документа (dc:language); інакше мова проєкту (рекомендований)	`language: uk-UA`	2.3
`doc_defaults.language`	project	language	мова документа (dc:language); інакше мова проєкту	`language: uk-UA`	10.6
`doc.identifier`	document	string	номер документа (користувацька властивість)	`identifier: "Д-2026-017"`	2.3
`doc_defaults.identifier`	project	string	номер документа (користувацька властивість)	`identifier: "Д-2026-017"`	10.6
`doc.revision`	document	string	редакція (користувацька властивість)	`revision: "2"`	2.3
`doc_defaults.revision`	project	string	редакція (користувацька властивість)	`revision: "2"`	10.6
`doc.status`	document	string	статус (користувацька властивість)	`status: "проєкт"`	2.3
`doc_defaults.status`	project	string	статус (користувацька властивість)	`status: "проєкт"`	10.6
`doc.created`	document	date	дата створення (meta:creation-date); інакше дата збирання (рекомендований)	`created: 2026-10-12`	2.3
`doc_defaults.created`	project	date	дата створення (meta:creation-date); інакше дата збирання	`created: 2026-10-12`	10.6
`doc.modified`	document	date	дата зміни (dc:date); інакше дата збирання	`modified: 2026-10-16`	2.3
`doc_defaults.modified`	project	date	дата зміни (dc:date); інакше дата збирання	`modified: 2026-10-16`	10.6
`doc.keywords`	document	strings	ключові слова (meta:keyword): рядок або список	`keywords: [договір, поставка]`	2.3
`doc_defaults.keywords`	project	strings	ключові слова (meta:keyword): рядок або список	`keywords: [договір, поставка]`	10.6
`params`	document, fragment, project	mapping	параметри зі значеннями за замовчуванням; ціль або --param їх перевизначає	`params: {vat: true, delivery_days: 5}`	10.6
`structure`	document	mapping	налаштування профілю структури документа	`structure: {numbering: {headings: false}}`	5.1
`styles`	document, project	paths	модулі стилів: список шляхів або один шлях (std/styles/default — вбудований)	`styles: [styles/business.s.prml]`	11.7
`references`	document	list	документи проєкту, на цілі яких посилається документ (імена з ref_documents проєкту)	`references: [contract]`	5.7
`defaults`	styles	mapping	типові властивості: шрифт, розмір, мова	`defaults: {font: Roboto, size: 11pt, language: uk-UA}`	11.1
`paragraphs`	styles	mapping	стилі абзаців	`paragraphs:`	11.2
`characters`	styles	mapping	стилі символів	`characters:`	11.2
`blocks`	styles	mapping	стилі блоків	`blocks:`	11.2
`tables`	styles	mapping	стилі таблиць	`tables:`	11.2

Ключ	Файли	Тип	Призначення	Приклад → результат	§
`lists`	styles	mapping	стилі списків	`lists:`	11.2
`frames`	styles	mapping	стилі врізок	`frames:`	11.2
`extends`	styles	path	базовий модуль стилів, підключається шаром перед цим (рішення 133)	`extends: std/styles/business`	11.7
`pages`	styles	mapping	стилі сторінок	`pages:`	11.2
`name`	project	string	назва проєкту	`name: "Договори 2026"`	10.6
`language`	project	language	типова мова документів	`language: uk-UA`	10.6
`extends`	project	path	базовий проєкт: його параметри, ключі, документи для посилань, стилі й типові метадані успадковуються	`extends: ../../project.p.prml`	10.6
`on_conflict`	project	string	конфлікти експорту фрагментів: error, first, last	`on_conflict: error`	10.4
`keys`	project	mapping	карта ключів: ім'я → шлях	`keys: {logo: assets/logo.svg}`	9.3
`limits`	project	mapping	ліміти ядра: time, recursion, size	`limits: {time: 20, recursion: 500, size: 10_000_000}`	12.8
`limits.time`	project	number	ліміт часу, секунди	`time: 20`	12.8
`limits.recursion`	project	int	ліміт глибини рекурсії	`recursion: 500`	12.8
`limits.size`	project	int	ліміт розміру значень	`size: 10_000_000`	12.8
`build_date`	project	date	фіксована дата збирання (today() і типові дати)	`build_date: 2026-10-12`	12.7
`targets`	project	list	цілі збирання primula build project.p.prml	`targets:`	10.7
`targets[].source`	project	path	вихідний документ	`- source: contract.prml`	10.7
`targets[].output`	project	path	вихідний ODT	`output: out/contract.odt`	10.7
`targets[].params`	project	mapping	значення параметрів цілі	`params: {vat: false}`	10.7
`targets[].keys`	project	mapping	ключі цілі	`keys: {logo: assets/other.svg}`	10.7
`targets[].pdf`	project	any	також PDF: true або його шлях	`pdf: true`	10.7
`ref_documents`	project	mapping	документи, на які посилаються інші (references: документа): ім'я: шлях або {source, registry}	`ref_documents: {contract: contract.prml}`	10.6

Додаток Г. Повідомлення транслятора

Згенеровано `tools/author_docs.py appendices` з довідника транслятора; не правити вручну.

Код	Вид	Повідомлення	§
P0001	error	'{command}' не реалізовано на етапі прототипу {stage}; результат не записується	— (временный, для команд прототипа)
P0002	error	внутрішня помилка транслятора: {detail}	14.2
P0004	error	перевищено ліміт часу збирання (--timeout)	12.8, 14.1 (--timeout)
P0005	error	бракує пам'яті: обчислення перевищило ліміт пам'яті процесу транслятора	12.8
P0006	error	перервано: обчислення введення зупинено	14.1 (primula console, решение 128)
P0101	error	не вдається прочитати файл '{path}': {reason}	14.1
P0102	error	файл не в кодуванні UTF-8: недопустимий байт у позиції {byte_offset}	2.1
P0103	error	не вдається записати файл '{path}': {reason}	14.3
P0105	error	--overlay '{text}' треба записувати як ФАЙЛ=БУФЕР, і файл буфера має існувати	14.1, решение 124
P0106	error	primula parallel: {problem}{value}	7.7, 14.1, решение 134
P0107	error	сегмент {segment}, потік '{track}' не в стані '{expected}': {reason} ({status})	7.7, 14.1, решение 134
P1101	error	заголовок YAML не закрито: бракує рядка '---'	2.3, A Header
P1102	error	помилка YAML у заголовку: {problem}	2.3
P1103	error	заголовок YAML має бути відображенням «ключ: значення», а знайдено {found}	2.3
P1104	warning	невідомий ключ '{key}' у заголовку файлу виду '{kind}'	2.3, 10.6, 11.1
P1105	error	ключ заголовка '{key}': очікується {expected}, а знайдено {found}	2.3, 10.6, 11.1
P1106	error	невідомий вид файлу '{value}'; очікується один із: {expected}	2.2
P1107	error	непідтримувана версія мови '{value}'; транслятор реалізує версію {supported}	0.1, 2.3
P1109	error	неіснуюча дата '{value}'	12.6 (решение 25)
P1110	error	повторний ключ '{key}'	YAML 1.2 (уникальность ключей)
P1111	error	непідтримуваний тег YAML '{tag}'	12.6 (схема core)
P1112	error	рекурсивний псевдонім YAML: значення містить саме себе	12.6
P1113	error	ключ відображення має бути скалярним значенням	12.6
P1114	warning	'{value}' у YAML 1.2 — рядок, а не логічне значення	12.6 (решение 25)
P1115	warning	'{value}' читається як ціле число {number}: початкові нулі втрачаються	12.6 (решение 25)
P1116	error	ключ 'lines' вилучено: режим рядків вилучено у версії 1.1, рядкова форма закінчується в кінці рядка	2.3, 3.4, решение 107
P1117	error	недопустима версія стандартної бібліотеки '{value}': очікується MAJOR.MINOR, наприклад "1.1"	2.3, 13

Код	Вид	Повідомлення	§
P1118	error	файл потребує стандартної бібліотеки {value}; транслятор має {available}	2.3, 13
P1119	error	часові пояси не підтримуються: '{value}'	2.3, 12.6, решение 109
P1120	error	ключ заголовка '{key}' перейменовано: пишіть '{new}:' (рішення 129)	2.3, 10.6, решение 129
P1201	error	після '|' має бути ім'я команди, '(' або коротка форма; знайдено {found}	3.1
P1202	error	невідома коротка форма '|{char}'	3.1, B.1
P1203	error	'|{char}' не є короткою формою	3.1
P1204	error	'{bracket}' не закрито	3.2, решение 22
P1205	error	закривна '{found}' не відповідає відкривній дужці; очікується '{expected}'	3.2
P1206	error	рядковий літерал не закрито	3.2
P1207	error	помилка синтаксису Python: {problem}	12.2
P1208	error	f-рядок містить вкладені лапки того самого виду; Primula використовує синтаксис Python 3.11	0.2, решение 30
P1209	error	'|{form}' потребує тіла '{{…}}'	3.3, A `InlineShort`
P1210	error	'|{form}' допустиме лише на початку рядка	3.3, B.1
P1211	error	одразу після імені команди '{name}' стоїть літера	3.2, A `InlineCommand`
P1212	error	після '|{form}' мають бути пробіл і текст	A `Heading`, `Clause`, `ListItem`
P1213	error	рівень заголовка {level} завеликий; допустимі рівні 1–6	5.2
P1214	error	'|!' потребує аргументів із видом примітки, наприклад |! ("warning")	9.1, A `Admonition`
P1215	error	'{{' не закрито	3.2, A `InlineBody`
P1216	error	'{name}' не закрито; очікується '{close}'	3.5
P1217	error	'{found}' не закриває '{name}'; очікується '{expected}'	3.5
P1218	error	'{found}': немає відкритого оточення, яке можна закрити	3.5
P1219	error	'|end' не може закрити '|{kind}'; використовуйте '|end{kind}'	3.5, решение 28
P1220	error	'|{form}' без відповідного '|if'	3.6, решение 21
P1221	error	маркер закриття має займати весь рядок	3.5, решение 34
P1222	error	рядок має менший або інший відступ, ніж рядок, що відкриває блок	3.7, решение 24
P1223	error	блок verbatim не закрито: немає рядка '{marker}'	3.7, решение 34
P1224	error	блок коду не закрито: немає рядка '|end'	2.2, 3.7
P1225	error	'|verbatim' приймає лише аргумент end="…" з непорожнім однорядковим рядком	3.7, A `VerbatimBlock`
P1226	error	'{what}' не допускається безпосередньо в '|{container}'	6.2, A 3.5
P1227	error	'|&' допустиме лише в рядку таблиці	6.4
P1228	error	формула '|=(…)' допустима лише в комірці таблиці	6.8
P1229	error	недопустимий заголовок макросу; очікується '|def ім'я(параметри):'	A `DefBlock`
P1230	error	заголовок не допускається в оточенні пункту	5.3
P1231	error	'{form}' не допускається в оточенні списку	5.4, решение 33
P1232	error	рівень списку {level} іде після рівня {previous}: рівень пропущено	5.4, решение 32

Код	Вид	Повідомлення	§
P1233	error	маркер списку '{marker}' не відповідає батьківським пунктам; очікується '{expected}'	5.4, решение 32
P1234	error	на рівні {level} одного списку змішано марковані й нумеровані пункти	5.4, решение 32
P1235	error	'{name}' — зарезервоване ім'я, його не можна визначати	B.3
P1236	warning	цей рядок починає новий абзац після рядкової форми	3.4, 14.2
P1238	warning	оточення завдовжки {lines} рядків закрито анонімним '|end'; напишіть '{close}'	3.5, 14.2, решение 35
P1239	error	після '|list(…)' немає списку	5.4
P1240	error	'|{form}' після '|else'	3.6, A `IfBlock`
P1241	error	оточення списку має починатися з пункту списку	5.4, решение 33
P1242	error	після '{after}' очікується {expected}	A
P1243	error	рядок таблиці '|:' допустимий лише в '|table'	6.4, A 3.5
P1244	error	'level=' не можна поєднувати з {count} '<': рівень задається або кількістю '<', або level= з одним '<'	3.3, 5.2, решения 105, 114
P1245	warning	у '|@(…)' немає тіла: текст у фігурних дужках після нього виводиться як є	5.7, решение 141
P2101	error	'{construct}' не допускається в коді Primula	12.2
P2102	error	ім'я '{name}' не допускається: імена з підкресленнями на початку ведуть до внутрішніх механізмів Python	12.2, 12.7
P2103	error	'is' та 'is not' допускаються лише з None	12.2
P2104	error	'{statement}' поза функцією або циклом	12.2
P2105	error	'{name} …=' на верхньому рівні повторно визначає глобальне ім'я; глобальні імена незмінні	12.7, решение 40
P2106	error	'@export' допускається лише як '@export' або '@export(name="…")' над def на верхньому рівні блоку коду	10.4, 12.2, решение 135
P2201	error	ім'я '{name}' не визначено	10.4
P2202	error	локальне ім'я '{name}' прочитано до присвоєння	12.7
P2203	error	'{name}' вже визначено	10.4
P2204	error	циклічне визначення: {chain}	10.4
P2301	error	операція '{op}' не підтримується для '{left}' і '{right}'	12.3
P2303	error	ділення на нуль	12.4
P2304	error	bool не бере участі в арифметиці; перетворіть через int(…)	12.4, решение 41
P2305	error	'{op}' не підтримується між '{left}' і '{right}'	12.3, решение 43
P2307	error	неможливо перетворити {value} на {type}	12.4, 12.10
P2308	error	недопустима числова операція ({detail})	12.4
P2309	error	переповнення числа: результат має понад {limit} значущих цифр	12.4, решение 108
P2310	error	часові пояси не підтримуються: {value}	12.3, решение 109
P2311	error	індекс {index} поза межами (розмір {size})	12.10
P2312	error	ключ {key} не знайдено	12.10
P2313	error	'{type}' не має атрибута '{name}'	12.10, решение 43
P2314	error	запис не має поля '{name}'	12.6
P2315	error	'{type}' не можна перебирати	12.10
P2316	error	'{type}' не підтримує '{operation}'	12.3
P2317	error	'{type}' не може бути ключем словника	12.3, решение 43
P2318	error	заморожене значення {type} не можна змінювати	12.7, решения 40, 46

Код	Вид	Повідомлення	§
P2319	error	{type} не можна змінювати під час перебору	12.7
P2320	error	неможливо розпакувати: очікується значень {expected}, отримано {got}	12.2
P2321	error	{function}(): {problem}	12.2
P2322	error	{function}: {problem}	12.10
P2324	error	неможливо присвоїти цій цілі	12.2
P2325	error	поле '{name}' задано двічі (імена порівнюються після нормалізації NFKC)	3.9, 12.6, решение 102
P2330	error	доступ до атрибута в полі формату не допускається: '{{{field}}}'; використовуйте індекс, наприклад {{0[name]}}	12.10, решение 45
P2331	error	недопустимий формат: {problem}	12.10
P2340	error	fail: {message}	12.9
P2402	error	перевищено ліміт рекурсії ({limit})	12.8
P2403	error	колекція з {size} елементів перевищує ліміт розміру ({limit})	12.8
P2404	error	рядок із {size} символів перевищує ліміт розміру ({limit})	12.8
P2406	error	перевищено ліміт часу одного обчислення ({limit} с)	12.8
P2501	warning	'{name}' присвоєно в операторі верхнього рівня, але там не використано; глобальним ім'ям воно не стає	12.7, решение 40
P2502	warning	ім'я '{name}' змішує абетки ({scripts})	3.9, 14.2, решение 102
P2503	warning	ім'я '{name}' відрізняється від '{other}' лише схожими літерами	3.9, 14.2, решение 102
P2601	error	невідома команда '|{name}': це не вбудована команда і не визначений макрос чи функція	3.2
P2602	error	'{name}' — значення типу {type}, а не команда	3.2
P2603	error	значення типу {type} не можна вставити в текст	4.2, решение 47
P2604	warning	значення типу {type} вставлено в текст без оформлення	4.2, 14.2, решение 47
P2605	error	вміст '{name}' має кілька блоків і не може бути вставлений усередині рядка	3.2, решение 48
P2606	error	макрос '{name}' не має параметра 'body', а виклик передає тіло	3.2, решение 48
P2608	error	'{what}' у гілці |if верхнього рівня: визначення не може бути умовним	10.4, решение 49
P2609	error	|{command}: {problem}	10.4
P2610	error	експортувати імена може лише фрагмент, підключений поза |if, |for, |def	10.4, решение 49
P2611	error	when= має бути логічним значенням, а знайдено {type}	9.2, решение 29
P2612	error	значення має бути відоме під час побудови дерева: '{name}' не є параметром заголовка	10.2, решение 50
P2613	error	недопустимий шлях '{path}': URL, абсолютні шляхи й шляхи поза коренем проєкту не допускаються	10.1
P2614	error	не вдається прочитати '{path}': {reason}	10.1
P2615	error	цикл підключень: {chain}	10.1; 11.7 — цикл основ `extends` модулей стилей (решение 133)
P2616	error	'{path}' — файл виду '{kind}', очікується '{expected}'	10.2, 10.3; 11.7 — основа `extends` не модуль стилей

Код	Вид	Повідомлення	§
P2617	error	невідомий параметр '{name}' фрагмента '{path}'	10.2
P2618	error	частину '{part}' не знайдено в '{path}'	10.2
P2619	error	невідомий ключ '{key}': його немає в карті ключів проєкту	9.3
P2620	error	'{name}' експортують два фрагменти; розведіть їх через ns= або rename=	10.4
P2621	error	'{name}' визначено в документі й експортовано фрагментом; використайте override=True або rename=	10.4
P2622	error	ім'я модуля '{name}' не є ідентифікатором або зарезервоване	10.3, решение 52
P2623	error	у модулі допускаються лише визначення: |def, |let, |code:, |import	10.3, решение 52
P2624	error	'{name}' не є публічним ім'ям модуля '{module}'	10.3, решение 52
P2625	error	модуль стандартної бібліотеки '{name}' на цьому етапі прототипу недоступний	13.1
P2626	error	параметр 'features' має бути списком рядків	9.2, решение 53
P2629	error	'{value}' у стовпці '{column}' не є коректним значенням типу {type}	10.5, решение 51
P2630	error	некоректні дані: {problem}	10.5, решения 25, 51
P2632	error	|call потребує функції першим аргументом	3.2
P2633	error	обчислений стиль має бути назвою стилю (рядком), а знайдено {type}	4.3
P2634	error	назва параметра '{name}' не є ідентифікатором	2.3
P2635	warning	у документі залишився заповнювач: '{text}'	4.2
P2636	error	заголовок отримує рівень {level} після level_shift={shift}; найглибший рівень — 6	10.2, 10.7, ПЗ
P2637	error	'{name}' не визначено у фрагменті, його не можна експортувати	10.4, решение 135
P2638	warning	'@export' у модулі нічого не змінює: кожне ім'я модуля без '_' на початку відкрите	10.3, решение 135
P2640	error	накопичувач '{name}' відрізняється у двох проходах: він залежить від власного результату (collected() прочитано до того, як collect() його змінив)	4.6, решение 143
P2639	warning	'{path}' лежить поза проєктом: символічне посилання '{link}' веде до '{target}'; зберігайте цей каталог разом із проєктом, щоб документ можна було зібрати знову	10.1, решение 136
P2701	warning	стандартна бібліотека не підтримує мову документа '{lang}'; використано англійські форми	библ. 1, решение 115
P2702	error	невідома мова '{lang}' у lang=; підтримуються: {allowed}	библ. 1, решение 115
P2703	error	{function}: число float не приймається в грошових розрахунках	библ. 3
P2704	error	{function}: значення {value} поза діапазоном ({range})	библ. 4, 5
P2705	error	невідома валюта '{code}'	библ. 3
P2706	error	невідома одиниця '{code}'	библ. 6
P2707	error	одиниці '{a}' і '{b}' вимірюють різні величини	библ. 6
P2708	error	{function}: відмінок '{value}' не підтримується; допустимі: {allowed}	библ. 4, 5
P2709	error	{function}: невідомий рід '{value}'; допустимі: {allowed}	библ. 4
P2710	error	fmt_date: шаблон показує час доби {placeholder}	библ. 5
P2711	error	{function}: невідомий заповнювач '{placeholder}' у шаблоні	библ. 5
P2712	error	{function}: невідома форма '{format}'; допустимі: {allowed}	библ. 3, 5

Код	Вид	Повідомлення	§
P2713	error	календар: невідомий ключ '{key}'; допустимі: {allowed}	библ. 5
P2714	error	календар: неправильне значення '{key}' (weekend: mon … sun; дати без часу доби)	библ. 5
P2715	error	календар: дата {date} одночасно робоча і вихідна	библ. 5, решение 115
P2716	error	plural: для дробового числа потрібна форма frac=	библ. 7, решение 115
P2717	error	{function}: крок має бути більшим за нуль	библ. 2
P2718	error	split: ваги — непорожній список невід'ємних чисел з додатною сумою	библ. 3
P2719	error	у валюти '{code}' немає форми frac для мови {lang}	библ. 1, 3, решение 115
P2720	error	qty: підтримується лише симетричний допуск (число в tol=)	библ. 6
P2721	error	{function}: аргумент '{arg}' має бути {expected}, а не {type}	библ. 1–7
P2722	error	{function}: для мови {lang} потрібно форм: {count}, задано: {given}	библ. 1, 3, 7
P2723	error	{function}: дробова частина має понад {places} знаків	библ. 4
P2724	error	{function}: елемент не має поля '{key}'	библ. 8
P2725	error	{function}: значення типу {type} не може бути ключем групи	библ. 8
P2727	error	{function}: аргумент {value} поза областю визначення функції	библ. 9
P2728	error	{function}: {size} байтів не вміщаються в QR-код із корекцією {ecc} до версії {version}	библ. 10, решение 137
P2729	error	{function}: символ '{char}' не можна записати в кодуванні {encoding}	библ. 10, решение 137
P3101	error	невідома властивість '{prop}' стилю виду {kind} '{name}'	11.3, решение 57
P3102	error	недопустиме значення '{prop}': очікується {expected}	11.3, решение 57
P3103	error	стиль '{name}' визначено двічі в одному шарі	11.7
P3104	error	'{name}' є водночас стилем абзацу і стилем символів	11.2
P3105	error	невідомий батьківський стиль '{parent}' стилю '{name}'	11.2
P3106	error	батьківський стиль '{parent}' стилю '{name}' має вид {kind}, очікується {expected}	11.2
P3107	error	цикл успадкування стилів: {chain}	11.2
P3108	error	невідомий стиль '{name}'	4.3
P3109	error	'{name}' — стиль виду {kind}; тут потрібен стиль виду {expected}	4.3
P3110	error	недопустима назва стилю '{name}': літера, далі літери, цифри, '_', '-'	11.2
P3111	error	назву стилю '{name}' вже має стиль з іншими властивостями	11.4, решение 56
P3112	error	невідомий крок шкали {scale} '{step}'	11.4
P3113	error	тут допустимі лише параметри before=, after=, indent=, а не '{name}'	11.4
P3114	error	тут не можна визначати стилі (верхній рівень модуля | import)	11.7, решение 59
P3115	error	невідома конструкція '{key}' для structure_styles / |set	4.5, 11.5, решение 60
P3118	warning	пряме форматування |fmt: зміна стилю у Writer не вплине на цей текст	4.4, 11.8, 14.2
P3120	error	невідомий профіль структури '{name}'; очікується business або report	5.1

Код	Вид	Повідомлення	§
P3121	error	стиль списку '{name}': формат '{format}' (кириличні літери) потребує документа українською або російською	11.3, решения 115, 133 (только для стилей, которые документ использует)
P3122	error	пункт списку {number} не має літери: рівень допускає {count} літер ('{letters}')	11.2, 5.4, П1
P4101	error	неприпустимий ID '{id}': латинська літера, далі літери, цифри, '_', '-', '.'	5.6
P4102	error	повторний ID '{id}'	5.6
P4103	error	фрагмент з ID '{id}' вставлено двічі без ns=	5.6, решение 74
P4104	error	невідомий аргумент '{name}' команди {command}	5
P4105	error	аргумент '{name}' команди {command} має бути {expected}	5
P4201	error	пропущено рівень заголовка: рівень {level} після рівня {previous}	5.2
P4202	warning	текст нумерованого заголовка починається з номера; номер додається автоматично	5.2
P4203	warning	підрозділ іде після пунктів розділу {number}: номери пунктів і підрозділів можуть збігтися	5.3, решение 68
P4204	error	пропущено рівень пункту: пункт рівня {level} без батьківського пункту	5.3
P4205	error	невідомий пункт '{id}' у continues= (пункт має йти раніше)	5.3
P4206	error	continues='{id}': ціль має бути пунктом того самого розділу, області й рівня	5.3
P4207	error	start= і continues= не можна використовувати разом	5.4
P4208	error	невідомий список '{name}' у continues= (список має йти раніше)	5.4
P4209	error	повторна назва списку '{name}'	5.4
P4210	error	повторна або зарезервована назва області '{name}'	5.5
P4211	error	літери додатка '{letter}' немає серед літер додатків '{letters}'	5.5, решение 71
P4212	error	рівень заголовка {value} не є цілим числом від 1 до 6	5.2, решение 114
P4301	error	невідома ціль посилання '{id}'	5.7
P4302	error	ціль посилання '{id}' виключено умовою	5.7, решение 75
P4303	warning	ціль посилання '{id}' виключено умовою {condition} (залишено через --review)	5.7, решение 75
P4304	error	неприйнятний режим посилання '{mode}' для цілі {kind} '{id}'	5.7, решение 76
P4305	error	немає міток посилань для мови документа '{language}'	5.7, решение 72
P4306	error	немає форми '{case}' мітки '{key}' для мови '{language}'	5.7, решение 72
P4307	error	невідомий режим посилання '{mode}'; очікується number, label, title або page	5.7
P4308	error	посилання на '{id}' іншого документа не може показувати номер сторінки (режим 'page')	5.7, решение 121
P4309	error	ціль '{id}' знайдено в кількох документах із references: {documents}	5.7, решение 121
P4401	error	невідомий вид переліку '{kind}'; очікується figure, table, abbr або index	5.9
P4402	error	неприпустиме налаштування структури {setting}: '{value}'; очікується {expected}	5.1–5.10
P5101	error	неприпустиме значення '{name}' у col(): '{value}'	6.3

Код	Вид	Повідомлення	§
P5102	error	неприпустима ширина колонки '{value}'; очікується "40mm", "30%", "*", "auto" або "20mm..60mm"	6.3
P5103	error	рядок {row}: клітинки займають позицій: {used}, вільних позицій: {free} (з урахуванням об'єднань)	6.5, решение 83
P5104	error	rows={rows} виходить за кінець таблиці (залишилося рядків: {remaining})	6.5
P5105	error	'{text}' не є значенням типу колонки {type}	6.8, решение 77
P5106	error	{name}() — функція формул таблиці; вона дозволена лише у формулі клітинки |=(…)	6.8, решение 81
P5107	error	невідома колонка таблиці '{column}'	6.8
P5108	error	рядок {row} поза таблицею (рядки 1–{rows})	6.8, решение 81
P5109	error	{function}(): значення типу {type} не є числом	6.8, решение 81
P5110	error	{function}() від порожнього списку значень	6.8, решение 81
P5111	error	цикл формул клітинок: {chain}	6.8
P5112	warning	метрики шрифту '{font}' недоступні (немає файлу шрифту або fontTools); ширини колонок наближені	6.3
P5113	warning	мінімальні ширини колонок {columns} не вміщуються; колонки звужено	6.3
P5114	error	формулу не можна записати як формулу Writer (live=True)	6.8, решение 82
P5115	error	|table(data=…): у рядку {row} немає поля '{field}'	6.2, П4
P5301	warning	сегмент {segment}: немає блоку потоку '{track}'; у комірці — заповнювач	7.7, решение 134
P5302	error	'|{command}' {where}	7.7, решение 134
P5303	error	'{what}' не допускається безпосередньо в '|{container}'	7.7, решение 134
P5304	error	'|{command}' не допускається всередині сегмента |parallel	7.7, решение 134
P5305	error	{command}: {problem}: {value}	7.7, решение 134
P5306	error	повторний ID сегмента '{id}'	7.7, решение 134
P5307	error	|track("{track}"): {problem}	7.7, решение 134
P5308	warning	стандартна бібліотека не підтримує мову '{lang}' потоку '{track}'; використовуються англійські форми	7.7, решение 134
P5310	warning	сегмент {segment}, потік '{track}': {found} там, де в основному потоці {expected}	7.7, решение 134
P5311	warning	сегмент {segment}, потік '{track}': числа й ідентифікатори відрізняються від основного потоку: бракує {missing}; зайві {extra}	7.7, решение 134
P5312	warning	сегмент {segment}, потік '{track}': ID відрізняються від основного потоку: бракує {missing}; зайві {extra}	7.7, решение 134
P5313	warning	сегмент {segment}, потік '{track}': цілі посилань відрізняються від основного потоку: бракує {missing}; зайві {extra}	7.7, решение 134
P5314	warning	сегмент {segment}, потік '{track}': імена в |(…) відрізняються від основного потоку: бракує {missing}; зайві {extra}	7.7, решение 134
P5315	warning	|parallel: потік '{track}' закінчується з лічильником «{kind}» {value}, основний потік — {primary}	7.7, решение 134
P5316	warning	виноски нумеруються заново щодо {restart}: Writer нумерує виноски потоків |parallel разом з основним потоком	7.7, 5.10, решение 134
P5320	warning	сегмент {segment}, потік '{track}': переклад застарів — основний блок змінено після підтвердження (rev {rev}, тепер {now})	7.7, решение 134
P5321	warning	сегмент {segment}, потік '{track}': підтверджений блок змінено після підтвердження (rev {rev}, тепер {now})	7.7, решение 134

Код	Вид	Повідомлення	§
P6101	error	не вдається використати зображення '{path}': {reason}	8.1, решение 85
P6102	error	неприпустиме значення '{name}' у {command}: '{value}'	7, 8
P6103	error	невідомий стиль сторінки '{name}'	7.2
P6104	error	зміна сторінки не дозволена всередині {where}	7.3
P6105	error	після зміни сторінки до кінця документа або оточення немає вмісту	7.3
P6106	error	{command} всередині групи, що не розривається	7.4
P6107	warning	rotate= поки не виводиться: зображення записано без повороту	8.5
P6108	warning	зображення '{path}' має орієнтацію EXIF {orientation}; її не застосовано — поверніть сам файл	8.2, решение 85
P6109	error	неприпустимий водяний знак '{value}': очікується текст або img(…)	8.6, решение 84
P6110	error	неприпустимий текст SVG у svg=: {reason}	8.2, решение 131
P6111	error	текст SVG у svg= не може містити оголошень DOCTYPE або ENTITY	8.2, решение 131
P6112	error	текст SVG у svg= посилається назовні на '{ref}': дозволені лише посилання '#…' і 'data:'	8.2, решение 131
P6113	error	|{command}: зображення задається рівно одним способом: шлях, значення img(…), key= або svg=	8.2, 8.3, решение 131
P7001	warning	kind '{kind}' суперечить підрозширенню '{name}', яке означає '{implied}'; діє ключ 'kind'	2.2, 14.2, решение 100
P7101	error	файл проєкту має містити 'kind: project' у заголовку, знайдено '{kind}'	10.6, решение 117
P7102	error	неприпустиме значення '{key}' у файлі проєкту: очікується {expected}	10.6, решение 117
P7103	error	build(): {problem}	10.7, решение 117
P7104	error	джерело '{path}' цілі збирання не можна використати: {reason}	10.7, решение 117
P7105	error	невідомий параметр '{name}' цілі збирання '{source}': його не оголошено ні в документі, ні в проєкті	10.6, 10.7, решение 117
P7106	error	параметр '{name}' отримує значення типу {type}, а оголошений з типом {expected}	10.6, 10.7, решение 117
P7107	error	ціль збирання перевизначає ключ '{key}', якого немає в карті ключів проєкту	9.3, 10.7, решение 117
P7108	error	результат '{path}' цілі збирання лежить поза каталогом проєкту	10.7, решение 117
P7109	error	результат '{path}' записують кілька цілей збирання	10.7, решение 117
P7110	error	параметр документа '{name}' має тип {type}, а однойменний параметр проєкту — тип {expected}	10.6, решение 117
P7111	error	{function}() доступна лише в секції коду файлу проєкту	10.6, 10.7, решение 117
P7112	error	немає карти ключів: key= потребує файлу проєкту з 'keys'	9.3, решение 117
P7113	error	ключ '{key}' вказує на частину файлу ('{value}'); '#частина' допустима лише в |include	9.3, решение 117
P7114	error	key= не можна поєднувати зі шляхом або part=	9.3, решение 117
P7115	warning	'{name}' експортують кілька фрагментів; on_conflict: {mode} залишає експорт {mode}	10.4, решение 117
P7116	error	|document допустима лише окремим рядком у тексті кореневого документа (зокрема всередині |if і |for), а не в оточенні, частині чи фрагменті	10.7, решения 117, 118

Код	Вид	Повідомлення	§
P7117	warning	ключ '{key}' у заголовку частини ігнорується: стилі й налаштування структури беруться з кореневого документа і проєкту	10.7, 14.2, решение 117
P7118	error	неприпустима або повторна область '{scope}' у |document	10.7, 5.6, решения 117, 118
P7119	error	документ '{path}' лежить поза каталогом проєкту '{project}'	10.6, 14.1, решение 117
P7120	error	{option} не можна використовувати під час збирання проєкту: результати задають цілі	14.1, решение 117
P7121	error	--dump-project потребує файлу проєкту (project.p.prml)	14.1, решение 117
P7122	warning	у проєкті немає цілей збирання	10.7, решение 117
P7123	error	--param '{text}' слід записати як ІМ'Я=ЗНАЧЕННЯ	14.1, П2
P7124	error	--param: документ '{path}' не має параметра '{name}' (ні власного, ні проєкту)	14.1, П2
P7125	error	--param не можна використовувати під час збирання проєкту: параметри цілей задаються в 'targets' і build()	14.1, 10.6, П2
P7126	error	references: '{name}' не є документом проєкту (ref_documents: {known})	10.6, решения 121, 129
P7127	error	документ '{name}', на який посилаються, не вдалося обчислити (помилок: {errors}); зберіть його окремо, щоб їх побачити	10.6, решение 121
P7128	error	документ '{name}', на який посилаються, не має ні джерела, ні реєстру ({registry})	10.6, решение 121
P7129	warning	джерело документа '{name}', на який посилаються, не знайдено: тексти посилань — з його реєстру {registry}	10.6, решение 121
P7130	error	extends: '{path}' утворює коло файлів проєкту	10.6, решение 123
P7131	error	extends: проєкт має лежати всередині кореня базового проєкту '{path}'	10.6, решение 123
P7132	error	--dump-structure і --dump-styles потребують документа, а не файлу проєкту	14.1, решение 124
P8001	warning	'{construct}' ще не виводиться (етап {stage}); його місце позначено заповнювачем	15
P8002	error	не вдається записати '{path}': {reason}	14.1
P8003	error	для експорту PDF потрібен LibreOffice: soffice або libreoffice не знайдено	14.1
P8004	error	експорт PDF не вдався: {reason}	14.1
P8006	warning	цей блок поки не можна записати всередині пункту списку, його пропущено	15
P8007	warning	|fmt: невідома або недопустима властивість '{prop}'	4.4
P8008	warning	номери сторінок у PDF (зміст, переліки, посилання на сторінки) не оновлено: {reason}	5.9, 14.1
P8101	warning	'{construct}' не має відповідника в експорті й пропускається (розмітка сторінок: колонтитули, номери сторінок)	14.1, решение 139
P8102	error	для експорту в цей формат потрібен pandoc: pandoc не знайдено (https://pandoc.org; --to json пише AST без нього)	14.1, решение 139
P8103	error	pandoc завершився з помилкою: {reason}	14.1, решение 139
P8201	error	для імпорту потрібен pandoc: pandoc не знайдено (https://pandoc.org)	14.1, решение 146
P8202	error	імпорт не вдався: {reason}	14.1, решение 146
P8203	error	невідомий формат джерела '{suffix}': вкажіть --from (docx, odt, rtf, gfm, html, epub, markdown) або відоме розширення	14.1, решение 146

Код	Вид	Повідомлення	§
P8204	error	'{file}' — старий файл Word: для перетворення на DOCX потрібен LibreOffice (soffice), його не знайдено	14.1, решение 146
P8205	error	'{file}' уже існує: він не замінюється (використайте --force або -o)	14.1, решение 146
P9101	error	невідомий вид блока '{kind}'; допустимі: {allowed}	9.1, решение 116
P9102	error	блок порожній: немає тексту, hazard= чи consequence=	9.1, решение 116
P9103	warning	'{kind}' без hazard=: зазначте вид і джерело небезпеки	9.1, 14.2, решение 116
P9104	warning	hazard= і consequence= для '{kind}' ігноруються	9.1, решение 116
P9105	warning	функція модуля стилів admonition() у версії мови 1.1 не використовується; блоки оформлюються стилями	9.1, решение 116
P9106	warning	цей |if визначає імена у гілках і в режимі --review не показується з усіма варіантами	9.2, решение 116
P9201	error	невідоме скорочення '{name}'	9.4, решение 116
P9202	error	скорочення '{name}' визначено з різними розшифруваннями	9.4, решение 116
P9203	error	show=False потребує розшифрування: |abbr("{name}", "…", show=False)	9.4, решение 116
P9204	warning	abbreviations(first="short") без |list_of("abbr"): розшифрування ніде не виводяться	9.4, решение 116
P9205	warning	|list_of("abbr"): у документі немає скорочень; перелік не виводиться	5.9, 9.4, решение 116
P9206	error	abbreviations({key}=…): неприпустиме значення '{value}'; допустимі: {allowed}	9.4, решение 116
P9207	error	|abbr потребує скорочення першим аргументом (рядок)	9.4, решение 116
P9301	error	|index потребує непорожнього терміна (рядок) і необов'язкового рядка sub=	9.5, решение 116

Додаток Д. Стилі комплекту

Згенеровано `tools/author_docs.py appendices` з довідника транслятора; не правити вручну.

std/styles/default

Вбудована основа: стилі конструкцій, підключається, якщо модулі стилів не названо.

Основа: немає. Типовий шрифт: Liberation Sans, 11pt.

Стиль	Тип	Властивості в модулі
`AbbreviationList`	table	frame: none, rules: none, padding: [0.5mm, 2mm, 0.5mm, 0mm], repeat_header: false, parent: TableGrid
`Admonition`	paragraph	keep_together: true, parent: Body
`AdmonitionCaution`	paragraph	bold: true, color: #9C6500, parent: AdmonitionText
`AdmonitionDanger`	paragraph	bold: true, color: #C00000, parent: AdmonitionText
`AdmonitionNotice`	paragraph	bold: true, color: #1F4E9E, parent: AdmonitionText
`AdmonitionTable`	table	frame: none, rules: none, padding: [1mm, 2mm, 1mm, 2mm], paragraph: AdmonitionText, may_break: false, space_before: 3mm, space_after: 3mm, parent: TableGrid
`AdmonitionTableCaution`	table	parent: AdmonitionTable
`AdmonitionTableDanger`	table	parent: AdmonitionTable
`AdmonitionTableNotice`	table	parent: AdmonitionTable
`AdmonitionTableWarning`	table	parent: AdmonitionTable
`AdmonitionText`	paragraph	align: left, space_after: 1mm, keep_with_next: false, parent: Body
`AdmonitionWarning`	paragraph	bold: true, color: #E36C09, parent: AdmonitionText
`AppendixDesignation`	paragraph	align: right, keep_with_next: true, space_after: 0mm, parent: Body
`AppendixSubtitle`	paragraph	align: right, keep_with_next: true, space_after: 2mm, parent: Body
`AppendixTitle`	paragraph	bold: true, align: center, keep_with_next: true, space_before: 2mm, space_after: 4mm, parent: Body
`Body`	paragraph	align: justify, line_height: 115%, space_after: 2mm, first_line: 0mm, widows: 2, orphans: 2
`BodyCentered`	paragraph	align: center, parent: Body
`BodyCompact`	paragraph	size: 10pt, line_height: 110%, space_after: 1mm, parent: Body
`BodyIndented`	paragraph	first_line: 10mm, parent: Body
`BodyLeft`	paragraph	align: left, parent: Body
`BodyRight`	paragraph	align: right, parent: Body
`BulletList`	list	levels: {1: {type: bullet, bullet: •, left: 5mm, hanging: 5mm}, 2: {type: bullet, bullet: –, left: 10mm, hanging: 5mm}, 3: {type: bullet, bullet: ◦, left: 15mm, hanging: 5mm}}
`Checklist`	list	paragraph: ListText, levels: {1: {type: bullet, bullet: ☐, left: 8mm, hanging: 5mm}}
`Clause`	paragraph	parent: Body
`Code`	character	font: Liberation Mono
`Contents1`	paragraph	align: left, space_after: 1mm, parent: Body
`Contents2`	paragraph	left: 5mm, parent: Contents1
`Contents3`	paragraph	left: 10mm, parent: Contents1
`Contents4`	paragraph	left: 15mm, parent: Contents1

Стиль	Тип	Властивості в модулі
`Contents5`	paragraph	left: 20mm, parent: Contents1
`Contents6`	paragraph	left: 25mm, parent: Contents1
`ContentsTitle`	paragraph	bold: true, size: 12pt, align: center, keep_with_next: true, space_after: 4mm, parent: Body
`DashList`	list	paragraph: ListText, levels: {1: {type: bullet, bullet: –, left: 7mm, hanging: 4mm}, 2: {type: bullet, bullet: –, left: 14mm, hanging: 4mm}}
`DefinedTerm`	character	bold: true
`Emphasis`	character	italic: true
`Endnote`	paragraph	parent: Footnote
`Figure`	paragraph	align: center, keep_with_next: true, space_before: 2mm, space_after: 1mm, parent: Body
`FigureCaption`	paragraph	align: center, space_before: 1mm, space_after: 2mm, parent: Body
`Footer`	paragraph	size: 9pt, align: center, space_after: 0mm, parent: Body
`Footnote`	paragraph	size: 9pt, align: left, line_height: 100%, space_after: 0mm, left: 4mm, first_line: -4mm, parent: Body
`Group`	block	keep_together: true
`Header`	paragraph	size: 9pt, align: center, space_after: 0mm, parent: Body
`Heading`	paragraph	bold: true, align: left, keep_with_next: true, space_before: 4mm, space_after: 2mm, parent: Body
`Heading1`	paragraph	size: 14pt, parent: Heading
`Heading2`	paragraph	size: 13pt, parent: Heading
`Heading3`	paragraph	size: 12pt, parent: Heading
`Heading4`	paragraph	size: 11pt, parent: Heading
`Heading5`	paragraph	size: 11pt, italic: true, parent: Heading
`Heading6`	paragraph	size: 11pt, bold: false, italic: true, parent: Heading
`IndexEntry`	paragraph	align: left, space_after: 1mm, parent: Body
`IndexEntry2`	paragraph	left: 5mm, parent: IndexEntry
`Landscape`	page	orientation: landscape, parent: Portrait
`LatinLetterList`	list	paragraph: ListText, levels: {1: {type: number, format: a, suffix:), left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`LetterList`	list	paragraph: ListText, levels: {1: {type: number, format: a, suffix:), left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`ListText`	paragraph	align: left, space_after: 1mm, parent: Body
`MixedList`	list	paragraph: ListText, levels: {1: {type: number, format: 1, suffix: ., left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`Note`	paragraph	size: 9pt, italic: true, align: left, parent: Body
`NoteAnchor`	character	position: super
`NoteLabel`	character	
`NoteList`	list	paragraph: Note, levels: {1: {type: number, format: 1, suffix: ., left: 5mm, hanging: 5mm}}
`NumberedList`	list	levels: {1: {type: number, format: 1, suffix: ., left: 6mm, hanging: 6mm}, 2: {type: number, format: 1, suffix: ., left: 12mm, hanging: 8mm}, 3: {type: number, format: a, suffix:), left: 18mm, hanging: 6mm}}
`ParallelHeader`	paragraph	bold: true, align: left, keep_with_next: true, parent: Body
`ParallelTable`	table	width: [percent, 100], frame: none, rules: none, padding: 0mm, repeat_header: true, space_before: 3mm, space_after: 3mm
`Placeholder`	character	background: #FFFF00
`Portrait`	page	size: A4, orientation: portrait, margin: [20mm, 15mm, 20mm, 25mm]

Стиль	Тип	Властивості в модулі
`Preformatted`	paragraph	font: Liberation Mono, size: 9pt, align: left, line_height: 100%, space_after: 0mm, widows: 1, orphans: 1, parent: Body
`Quote`	paragraph	italic: true, left: 7mm, right: 5mm, space_before: 2mm, space_after: 3mm, parent: Body
`Reference`	character	
`ReviewFalse`	character	background: #F8D7D7
`ReviewLabel`	paragraph	size: 8pt, italic: true, color: #555555, align: left, space_after: 0mm, keep_with_next: true, parent: Body
`ReviewTrue`	character	background: #D9F2D9
`SmallPrint`	paragraph	align: left, size: 9pt, line_height: 110%, space_after: 1.5mm, color: #333333, parent: Body
`SmallText`	character	size: 9pt
`StrikeText`	character	strike: true
`Strong`	character	bold: true
`SubClause`	paragraph	left: 5mm, parent: Clause
`Subscript`	character	position: sub
`Superscript`	character	position: super
`TableCaption`	paragraph	align: left, keep_with_next: true, space_before: 2mm, space_after: 1mm, parent: Body
`TableCompact`	table	padding: 0.8mm, paragraph: TableTextCompact, header_paragraph: TableHeaderCompact, parent: TableGrid
`TableGrid`	table	width: [percent, 100], frame: 0.5pt solid #000000, rules: all, padding: 1mm, paragraph: TableText, header_bold: true, repeat_header: true
`TableHeader`	paragraph	bold: true, parent: TableText
`TableHeaderCompact`	paragraph	bold: true, parent: TableTextCompact
`TableNoBorders`	table	frame: none, rules: none, padding: [1mm, 2mm, 1mm, 0mm], repeat_header: false, header_background: none, parent: TableGrid
`TableText`	paragraph	align: left, space_after: 0mm, parent: Body
`TableTextCompact`	paragraph	size: 9pt, line_height: 105%, parent: TableText
`Underline`	character	underline: true

std/styles/business

Договори, акти, рахунки, протоколи; поля й відступи — за ДСТУ 4163:2020.

Основа: `std/styles/default`. Типовий шрифт: Liberation Serif, 12pt.

Стиль	Тип	Властивості в модулі
`Approval`	paragraph	left: 100mm, space_after: 0mm, keep_with_next: true, keep_together: true, parent: BodyLeft
`ApprovalTitle`	paragraph	bold: true, space_after: 1mm, parent: Approval
`AttachmentLabel`	paragraph	size: 10pt, space_before: 3mm, keep_with_next: true, parent: BodyLeft
`Body` (доповнює)	paragraph	align: justify, line_height: 115%, space_after: 2mm
`ClauseContinuation`	paragraph	left: 7mm, parent: Body
`DatePlace`	paragraph	space_before: 2mm, space_after: 4mm, keep_with_next: true, parent: BodyLeft
`Details`	paragraph	size: 10pt, line_height: 110%, space_after: 1mm, parent: BodyLeft
`DetailsTitle`	paragraph	bold: true, size: 11pt, space_after: 2mm, keep_with_next: true, parent: Details
`DocumentSubtitle`	paragraph	space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyCentered

Стиль	Тип	Властивості в модулі
`DocumentTitle`	paragraph	size: 14pt, bold: true, space_before: 3mm, space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyCentered
`DocumentVersion`	paragraph	size: 9pt, color: #444444, space_after: 4mm, keep_with_next: true, parent: BodyCentered
`FigureCaption` (доповнює)	paragraph	space_after: 3mm, size: 10pt
`Footer` (доповнює)	paragraph	color: #333333
`FormLabel`	paragraph	bold: true, space_after: 1mm, keep_with_next: true, parent: BodyLeft
`Header` (доповнює)	paragraph	color: #333333
`Heading` (доповнює)	paragraph	align: left, keep_together: true
`Heading1` (доповнює)	paragraph	size: 12pt, space_before: 4mm, space_after: 2mm
`Heading2` (доповнює)	paragraph	size: 12pt, space_before: 3mm, space_after: 1.5mm
`Heading3` (доповнює)	paragraph	size: 12pt, space_before: 2mm, space_after: 1mm
`Heading4` (доповнює)	paragraph	size: 12pt, italic: true, space_before: 2mm, space_after: 1mm
`Illustration`	frame	width: 110mm
`IllustrationSmall`	frame	width: 70mm
`IllustrationWide`	frame	width: 160mm
`Logo`	frame	width: 40mm
`LogoSmall`	frame	width: 25mm
`Note` (доповнює)	paragraph	size: 10pt, italic: false, space_before: 1mm
`OrganizationName`	paragraph	bold: true, space_after: 2mm, keep_with_next: true, parent: BodyLeft
`Placeholder` (доповнює)	character	background: #FFF2CC
`Portrait` (доповнює)	page	margin: [20mm, 10mm, 20mm, 30mm], footer_gap: 3mm, footer_min_height: 5mm
`Preamble`	paragraph	space_after: 3mm, parent: Body
`SignatureLine`	paragraph	space_after: 3mm, keep_together: true, parent: BodyLeft
`SignatureRole`	paragraph	space_before: 5mm, space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyLeft
`TableCaption` (доповнює)	paragraph	space_before: 3mm, space_after: 1.5mm, size: 10pt
`TableForm`	table	cols: [30%, 70%], parent: TableGrid
`TableGrid` (доповнює)	table	frame: 0.5pt solid #999999, padding: 1.5mm, border: 0.5pt solid #999999, header_background: #EEEEEE, header_paragraph: TableHeader, keep_rows: true, space_before: 1mm, space_after: 3mm
`TableParties`	table	cols: [50%, 50%], paragraph: Details, header_paragraph: DetailsTitle, header_background: none, padding: [0.5mm, 3mm, 0.5mm, 0mm], may_break: false, parent: TableNoBorders
`TableSchedule`	table	cols: [35%, 45%, 20%], parent: TableGrid
`TableServices`	table	cols: [52%, 12%, 18%, 18%], parent: TableGrid
`TableText` (доповнює)	paragraph	size: 10pt, line_height: 110%, widows: 1, orphans: 1

std/styles/letter

Службові листи за ДСТУ 4163:2020.

Основа: `std/styles/business`. Типовий шрифт: Liberation Serif, 12pt.

Стиль	Тип	Властивості в модулі
`BodyIndented` (доповнює)	paragraph	space_after: 1mm
`Executor`	paragraph	space_before: 8mm, space_after: 0mm, keep_together: true, parent: SmallPrint
`LetterFirst`	page	next: Portrait, parent: Portrait

Стиль	Тип	Властивості в модулі
`LetterSubject`	paragraph	bold: true, space_before: 6mm, space_after: 4mm, right: 97mm, keep_with_next: true, parent: BodyLeft
`LetterheadDetails`	paragraph	size: 9pt, line_height: 105%, space_after: 0mm, right: 97mm, parent: BodyLeft
`LetterheadName`	paragraph	bold: true, size: 12pt, space_after: 1mm, right: 97mm, parent: BodyLeft
`OutgoingNumber`	paragraph	size: 11pt, space_before: 4mm, space_after: 0mm, right: 97mm, parent: BodyLeft
`Recipient`	paragraph	left: 90mm, line_height: 100%, space_after: 1.5mm, keep_with_next: true, parent: BodyLeft
`Salutation`	paragraph	space_after: 3mm, keep_with_next: true, parent: BodyCentered

std/styles/technical

Настанови, інструкції, технічні описи.

Основа: `std/styles/default`. Типовий шрифт: Liberation Sans, 10.5pt.

Стиль	Тип	Властивості в модулі
`Body` (доповнює)	paragraph	align: left, line_height: 120%, space_after: 2mm
`Code` (доповнює)	character	background: #F2F2F2
`Command`	character	font: Liberation Mono, bold: true
`DefinitionTerm`	paragraph	bold: true, space_after: 0.5mm, keep_with_next: true, parent: Body
`DefinitionText`	paragraph	left: 8mm, space_after: 2mm, parent: Body
`FileName`	character	font: Liberation Mono
`Heading1` (доповнює)	paragraph	size: 15pt, space_before: 6mm, space_after: 3mm
`Heading2` (доповнює)	paragraph	size: 13pt, space_before: 5mm, space_after: 2mm
`Heading3` (доповнює)	paragraph	size: 11.5pt, space_before: 4mm, space_after: 1.5mm
`Key`	character	font: Liberation Mono, bold: true, background: #E8E8E8
`MenuItem`	character	bold: true
`Portrait` (доповнює)	page	margin: [20mm, 15mm, 20mm, 25mm]
`Preformatted` (доповнює)	paragraph	size: 9pt, space_after: 2mm, left: 3mm, right: 3mm, background: #F2F2F2, space_before: 1mm
`ProgramOutput`	paragraph	color: #333333, parent: Preformatted
`StepList`	list	paragraph: ListText, levels: {1: {type: number, format: 1, suffix: ., left: 8mm, hanging: 8mm}, 2: {type: number, format: a, suffix:), left: 16mm, hanging: 6mm}}
`StepResult`	paragraph	italic: true, left: 8mm, space_after: 2mm, parent: Body
`TableGrid` (доповнює)	table	frame: 0.5pt solid #808080, border: 0.5pt solid #808080, header_background: #E6E6E6, header_paragraph: TableHeader
`TableRevisions`	table	cols: [16mm, 24mm, *, 40mm], parent: TableCompact
`Variable`	character	font: Liberation Mono, italic: true

std/styles/report

Звіти за ДСТУ 3008:2015.

Основа: `std/styles/default`. Типовий шрифт: Liberation Serif, 14pt.

Стиль	Тип	Властивості в модулі
`AppendixDesignation` (доповнює)	paragraph	align: center, first_line: 0mm, bold: true
`AppendixSubtitle` (доповнює)	paragraph	align: center, first_line: 0mm

Стиль	Тип	Властивості в модулі
`AppendixTitle` (доповнює)	paragraph	bold: false, first_line: 0mm
`Body` (доповнює)	paragraph	align: justify, line_height: 150%, space_after: 0mm, first_line: 12.5mm
`Contents1` (доповнює)	paragraph	space_after: 0mm, first_line: 0mm
`ContentsTitle` (доповнює)	paragraph	parent: StructuralHeading
`Figure` (доповнює)	paragraph	first_line: 0mm
`FigureCaption` (доповнює)	paragraph	space_before: 1mm, space_after: 3mm, first_line: 0mm
`Footer` (доповнює)	paragraph	first_line: 0mm
`Footnote` (доповнює)	paragraph	size: 12pt
`Header` (доповнює)	paragraph	size: 12pt, align: right, first_line: 0mm
`Heading` (доповнює)	paragraph	align: left, space_before: 7mm, space_after: 7mm, first_line: 12.5mm, keep_together: true
`Heading1` (доповнює)	paragraph	size: 14pt, odf: {fo:text-transform: uppercase}
`Heading2` (доповнює)	paragraph	size: 14pt
`Heading3` (доповнює)	paragraph	size: 14pt
`Heading4` (доповнює)	paragraph	size: 14pt, italic: false
`ListText` (доповнює)	paragraph	space_after: 0mm, first_line: 0mm, line_height: 150%
`Note` (доповнює)	paragraph	size: 12pt, italic: false, align: justify, line_height: 100%, first_line: 12.5mm
`ParallelHeader` (доповнює)	paragraph	first_line: 0mm
`Portrait` (доповнює)	page	margin: [20mm, 10mm, 20mm, 25mm]
`Preformatted` (доповнює)	paragraph	size: 12pt, first_line: 0mm
`ReportTitle`	page	next: Portrait, parent: Portrait
`StructuralHeading`	paragraph	align: center, first_line: 0mm, page_break_before: true, parent: Heading1
`TableCaption` (доповнює)	paragraph	space_before: 3mm, space_after: 1mm, first_line: 12.5mm
`TableText` (доповнює)	paragraph	first_line: 0mm, size: 12pt, line_height: 100%
`TitlePageName`	paragraph	bold: true, space_before: 40mm, space_after: 6mm, odf: {fo:text-transform: uppercase}, parent: TitlePageText
`TitlePageText`	paragraph	first_line: 0mm, space_after: 2mm, parent: BodyCentered

std/styles/compact

Пам'ятки, довідки, шпаргалки.

Основа: `std/styles/default`. Типовий шрифт: Liberation Sans, 9pt.

Стиль	Тип	Властивості в модулі
`Body` (доповнює)	paragraph	align: left, line_height: 112%, space_after: 1mm, widows: 1, orphans: 1
`BulletList` (доповнює)	list	levels: {1: {type: bullet, bullet: •, left: 4mm, hanging: 3mm}, 2: {type: bullet, bullet: –, left: 8mm, hanging: 3mm}, 3: {type: bullet, bullet: ◦, left: 12mm, hanging: 3mm}}
`Footer` (доповнює)	paragraph	size: 7.5pt
`Header` (доповнює)	paragraph	size: 7.5pt
`Heading` (доповнює)	paragraph	space_before: 2.5mm, space_after: 1mm
`Heading1` (доповнює)	paragraph	size: 12pt
`Heading2` (доповнює)	paragraph	size: 10.5pt
`Heading3` (доповнює)	paragraph	size: 9.5pt
`Heading4` (доповнює)	paragraph	size: 9pt

Стиль	Тип	Властивості в модулі
`Heading5` (доповнює)	paragraph	size: 9pt
`Heading6` (доповнює)	paragraph	size: 9pt
`ListText` (доповнює)	paragraph	space_after: 0.5mm
`Note` (доповнює)	paragraph	size: 8pt
`Portrait` (доповнює)	page	margin: [12mm, 12mm, 12mm, 12mm]
`Preformatted` (доповнює)	paragraph	size: 8pt, background: #F2F2F2, left: 1mm, right: 1mm
`SmallPrint` (доповнює)	paragraph	size: 7.5pt
`TableGrid` (доповнює)	table	frame: 0.4pt solid #808080, padding: 0.7mm, border: 0.4pt solid #808080, header_background: #E6E6E6, header_paragraph: TableHeader
`TableText` (доповнює)	paragraph	size: 8.5pt, line_height: 105%
`TableTextCompact` (доповнює)	paragraph	size: 8pt

Глоссарий терминов Primula (ru / uk / en)

Единые термины руководства автора, справки Studio, сообщений транслятора и переводов. Переводчик (человек или модель по зафиксированному промпту этапа 15.5) берёт термин отсюда, а не подбирает синоним. Новый термин сначала вносится сюда. Записи `|…` — конструкции языка, они не переводятся.

ru	uk	en	Пояснение
документ	документ	document	файл `*.prml`, собирается в ODT
фрагмент	фрагмент	fragment	файл `*.f.prml`, подключается `|include`
модуль	модуль	module	файл `*.m.prml`, только определения, `|import`
модуль стилей	модуль стилів	styles module	файл `*.s.prml`
проект	проєкт	project	файл `project.p.prml`: параметры, ключи, цели
часть (сводного документа)	частина	part	`|document` или `|part`
заголовок YAML	заголовок YAML	YAML header	блок между строками `---` в начале файла
ключ заголовка	ключ заголовка	header key	`doc:`, `params:`, `styles:` …
метаданные	метадані	metadata	ключ `doc:`; в проекте `doc_defaults:`
параметр	параметр	parameter	`params:`, `--param`
карта ключей	карта ключів	key map	`keys:` проекта для `key=`
цель (сборки)	ціль	target	`targets:` проекта, `build()`
сборка	збирання	build	`primula build`
проверка	перевірка	check	`primula check`
транслятор	транслятор	translator	программа `primula`
разметка	розмітка	markup	текст с командами `|`
команда	команда	command	`|имя(…)`
длинная форма	довга форма	long form	`|имя(…)`, `|имя(…):`
краткая форма	коротка форма	short form	`|>`, `|*{…}` …
строчная форма	рядкова форма	line form	команда до конца строки
окружение	оточення	environment	`|имя(…):` … `|end имя`
маркер закрытия	маркер закриття	closing marker	`|end`, `|endif` …
аргумент	аргумент	argument	значение в `(…)` команды
тело	тіло	body	`{…}` команды или окружение
выражение	вираз	expression	`|(…)`, аргументы команд
блок кода	блок коду	code block	`|code:` … `|end`
вычислительное ядро	обчислювальне ядро	core (evaluator)	подмножество Python
определение	визначення	definition	имя, заданное кодом, `|let`, `|def`
макрос	макрос	macro	`|def` … `|enddef`
действие	дія	action	оператор верхнего уровня кода, кроме определения: `for`, `if`, вызов
запись	запис	record	значение `record`: поля через точку
экспорт	експорт	export	передача имени фрагмента документу: `@export`, `|export`, `export=True`
декоратор	декоратор	decorator	`@export` над `def`

ru	uk	en	Пояснение
стандартная библиотека	стандартна бібліотека	standard library	функции `num`, `money`, `fmt_date` …
библиотека	бібліотека	library	каталог модулей и данных для нескольких проектов (11)
символическая ссылка	символічне посилання	symbolic link	`lib` → `../shared/`…: подключение библиотеки (решение 136)
документ-тест	документ-тест	test document	документ, который проверяет библиотеку через `fail()`
заголовок (раздела)	заголовок	heading	`|<` … `|<<<<<<`
пункт	пункт	clause	`|>`, нумеруется по уровню
элемент списка	елемент списку	list item	`|-`, `|#`
предупреждающий блок	застережний блок	admonition	`|!("warning")`
приложение (к документу)	додаток	appendix	`|appendix`
перекрёстная ссылка	перехресне посилання	cross reference	`|@("id")`
идентификатор	ідентифікатор	ID	`id=` цели ссылки
сноска	виноска	footnote	`|footnote`
оглавление	зміст	table of contents	`|toc`
таблица	таблиця	table	`|table`
строка (таблицы)	рядок	row	`|:`
ячейка	клітинка	cell	`|&`
столбец	стовпець	column	`cols=`
шапка таблицы	шапка таблиці	header rows	`header=`
итоговые строки	підсумкові рядки	footer rows	`footer=`
формула ячейки	формула клітинки	cell formula	`|=(…)`
стиль	стиль	style	имя в модуле стилей
абзацный стиль	стиль абзацу	paragraph style	`|.Имя:`
символьный стиль	символьний стиль	character style	`|.Имя{…}`
стиль страницы	стиль сторінки	page style	`page_style`, `|page`
слой стилей	шар стилів	style layer	проект → `--style` → документ
набор стилей (комплекта)	набір стилів	styles set	`std/styles/business` … (раздел 11.9)
основа модуля стилей	основа модуля стилів	base (of a styles module)	`extends:` модуля стилей
раздел с колонками	розділ з колонками	section	`|section(cols=…)`
параллельные тексты	паралельні тексти	parallel texts	`|parallel`
поток (параллельного текста)	потік	track	`track(…)`, `|track`
основной поток	основний потік	primary track	`primary=`
сегмент	сегмент	segment	`|segment`
сегмент на всю ширину	сегмент на всю ширину	full-width segment	`span="all"`
проверка соответствия	перевірка відповідності	correspondence check	`parity=`
подтверждение перевода	підтвердження перекладу	confirmation of a translation	`primula parallel --stamp`, `rev=`
перевод устарел	переклад застарів	the translation is outdated	P5320
колонтитул	колонтитул	header / footer	`|header`, `|footer`

ru	uk	en	Пояснение
рисунок	рисунок	figure	`|figure`
изображение	зображення	image	`|img`, `img(…)`
водяной знак	водяний знак	watermark	`|watermark`
заполнитель	заповнювач	placeholder	`|?{…}`
буквальный текст	буквальний текст	verbatim text	`|verbatim`
комментарий	коментар	comment	`|%`, `|comment`
сообщение (диагностики)	повідомлення	diagnostic	`error[P1209]`
ошибка / предупреждение	помилка / попередження	error / warning	серьёзность сообщения
консоль вычислителя	консоль обчислювача	evaluator console	`primula console`